计算机操作系统

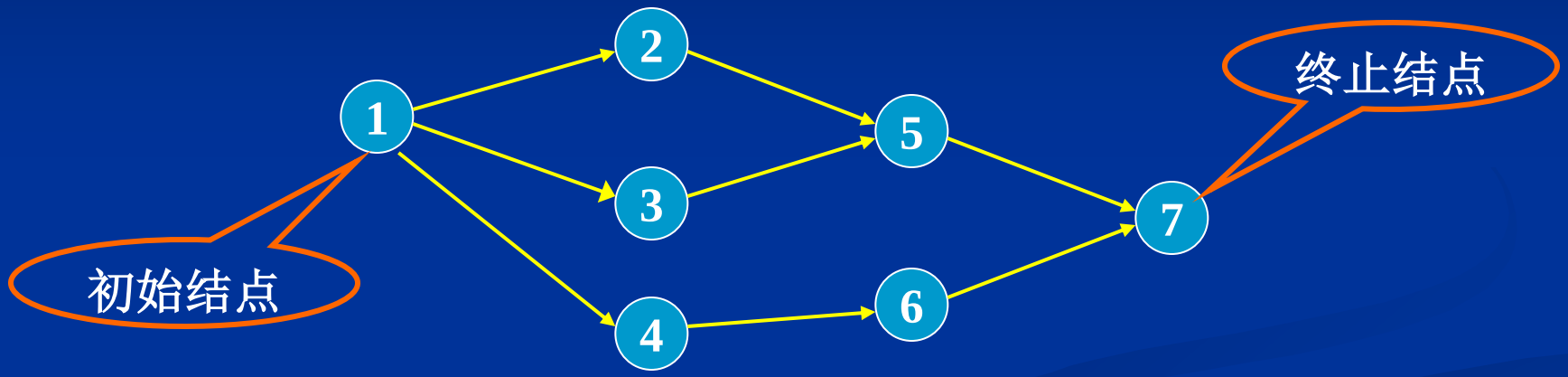
Operating Systems

田卫东

March, 2014

第2章 进程管理

2.1 进程的基本概念
2.1.1 前趋图



结点 ：代表一条语句，一段程序或一个进程

有向边：两结点之间存在的先后次序关系➔

➔ ={ (Pi,Pj) | Pi must complete before Pj may start }

图中：

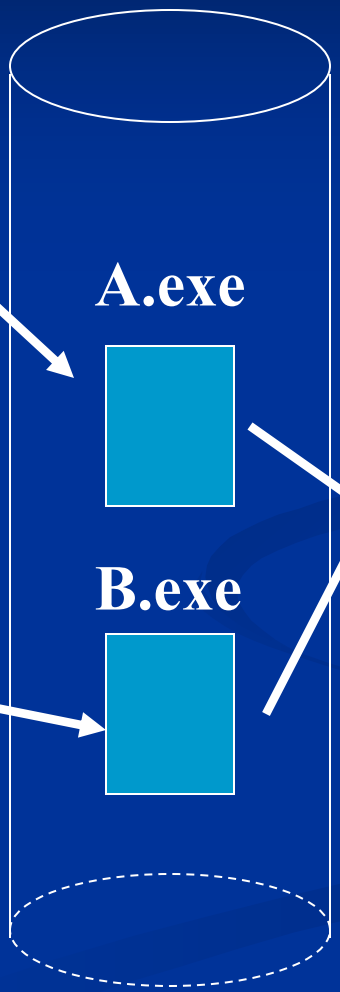
P = { P1, P2, P3, P4, P5 ,P6,P7}

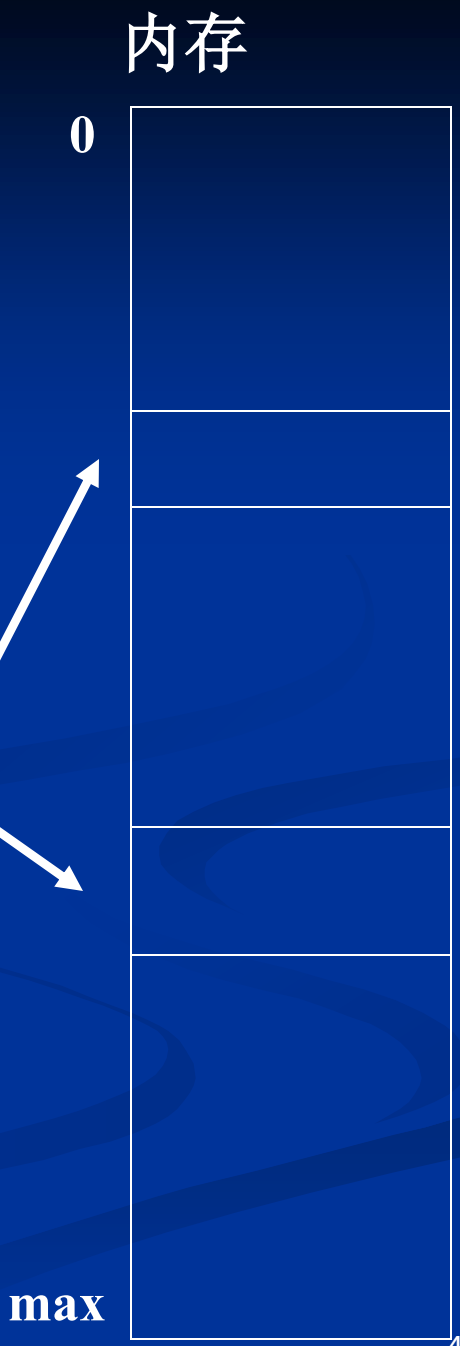
➔ = { (P1,P2), (P1,P3), (P1,P4), (P2,P5), (P3,P5),
(P4,P6), (P5,P7), (P6,P7) }

2.1 进程的基本概念

```
#include <stdio.h>
int main()
{
    printf("hello world");
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int a = 10*10*3.14;
    return 0;
}
```



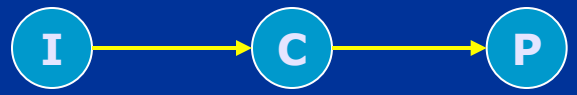


2.1 进程的基本概念
2.1.2 程序的顺序执行

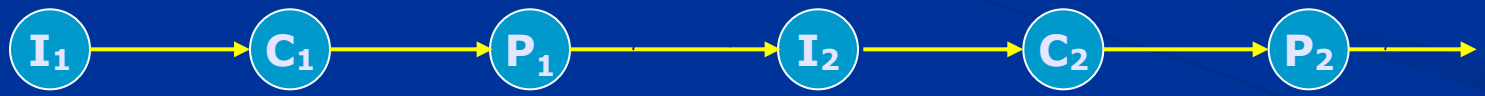
(1) 顺序执行的基本概念

■程序段级顺序执行

- 输入程序段：I
- 计算程序段：C
- 打印程序段：P



■程序级顺序执行



■语句级顺序执行

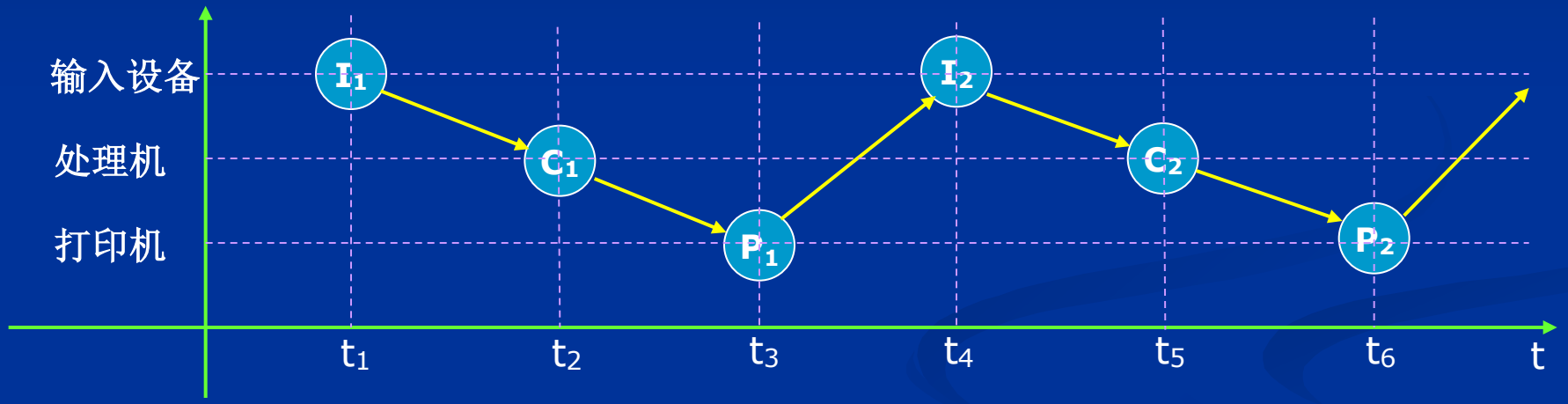
S1: a := x + y
S2: b := a – 5
S3: c := b + 1



(2) 顺序执行的特征

- **顺序性**
  处理机严格按照程序规定的顺序执行程序
- **封闭性**
  程序独占全机资源，不受其他程序的影响
- **可再现性**
  程序执行结果 = 初始条件 + 执行时的环境

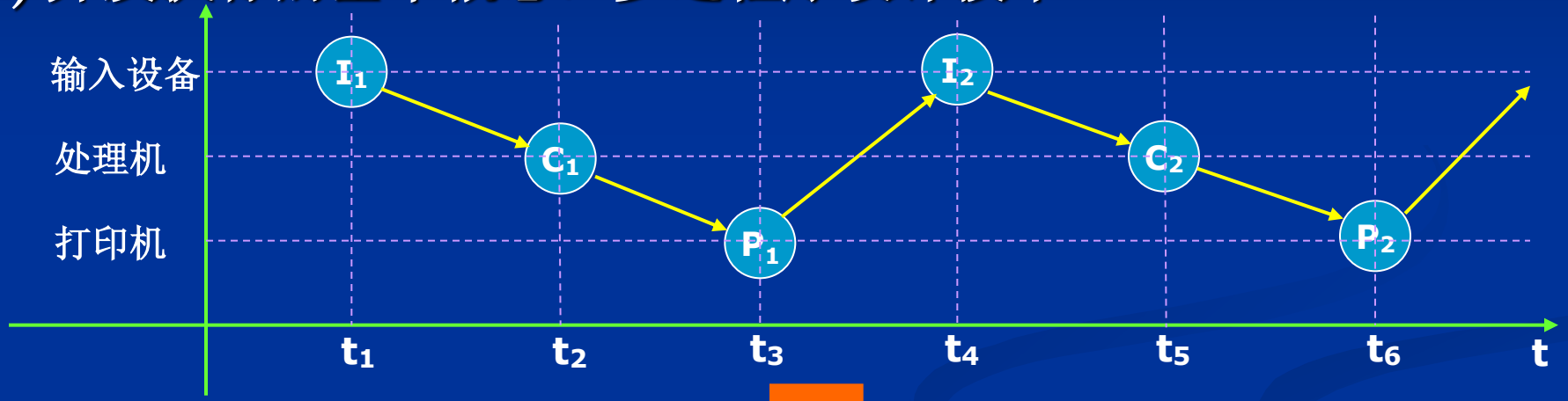
(3) 顺序执行存在的问题

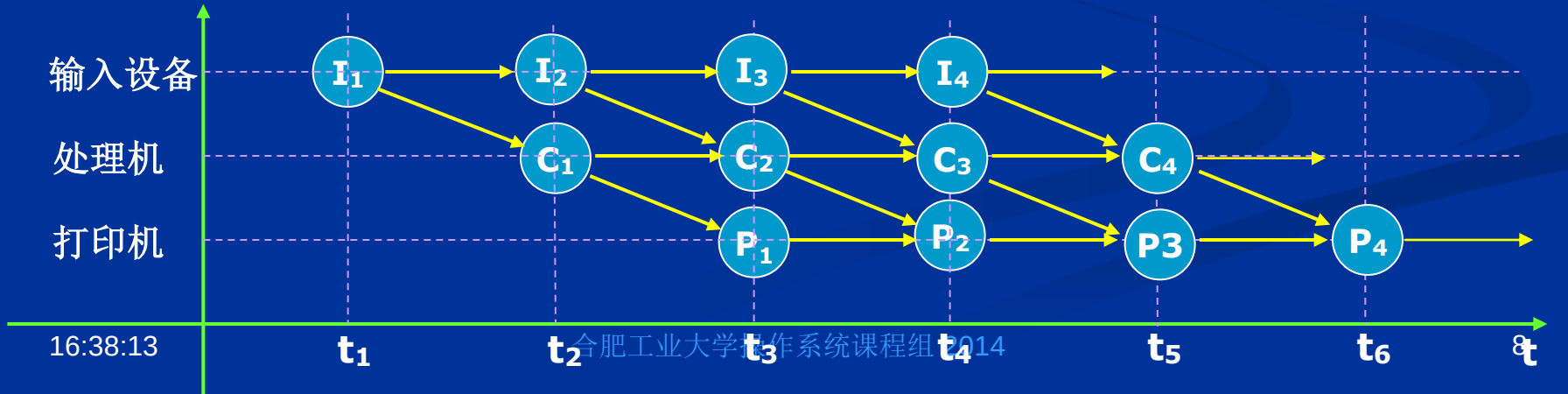


- 存在的问题
  - 整体资源利用率低
  - 单位时间内算题量少

2.1 进程的基本概念
2.1.3 程序的并发执行

(1) 并发执行的基本概念：多道程序设计技术





(2) 并发执行的问题

```
VAR  N : integer = 3 ;
Program A :
Begin
      while ( TRUE )
        N : = N+1 ;
      end while
end
```

程序A

```
Program B :
Begin
      while ( TRUE )
          print(N) ;
          N := 0 ;
      end while
end
```

程序B

语句执行顺序：		程序B打印结果
N := N +1 Print(N); N:=0	→	4
Print(N); N := N +1 N:=0	→	3
N := N +1 N:=0 Print(N);	→	0

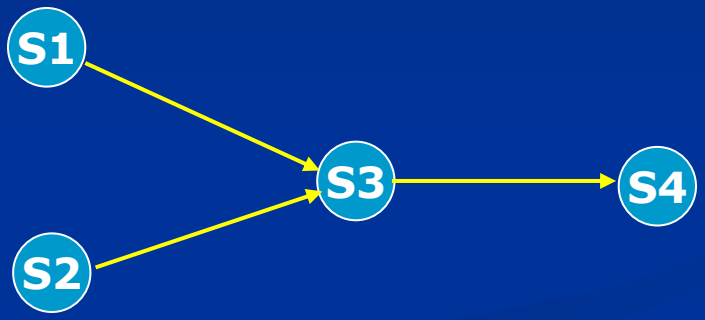
结论：

程序B因为受到程序A的影响，运行结果出现了不可再现性，即结果不定。

(3) 什么条件下，允许程序的并发执行？

- 请研究下面典型程序段的并发执行：

```
S1:    a := x + 2
S2:    b := y + 1
S3:    c := a + b
S4:    d := c + 6
```



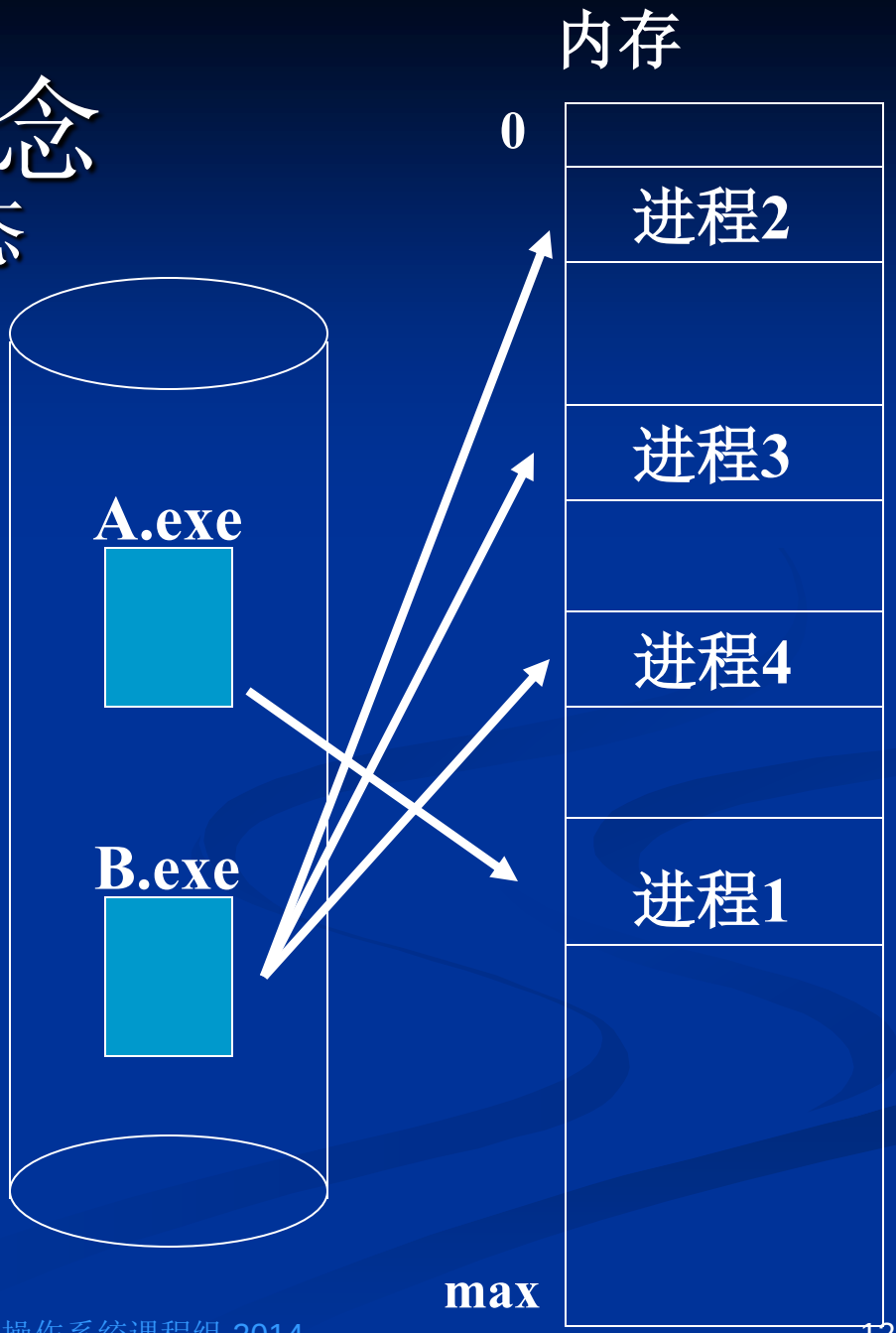
(4) 并发执行的特征

- **间断性**
  由于资源的共享，程序执行具有"执行-暂停-执行"的特点
- **失去封闭性**
  程序不再独占全机资源，运行受到其他程序的影响
- **不可再现性**
  程序执行结果受其他程序的影响，结果不定

2.1 进程的基本概念
2.1.4 进程的特征与状态

(1) 进程概念的产生

- 在多道程序环境下，程序的并发执行代替了程序的顺序执行，程序的活动不再处于封闭系统中，从而出现了许多新特征。为此人们引入了新概念—进程



(2) 进程的特征

- 对进程的解释：
  - 程序在处理机上的一次执行过程；
  - 可以和别的计算并行执行的计算；
  - 进程是程序在一个数据集合上运行的过程，是系统进行资源分配和调度的一个独立单位；
  - 是一个具有一定功能的程序，是关于某个数据集合的一次运行活动。

(2) 进程的特征

- 动态性

  是程序的一次执行过程，因而是动态的。

- 并发性

  引入进程就是为了和其他程序并发执行，提高资源利用率。

- 独立性

  能独立运行的基本单位，也是资源分配和调度的基本单位。

- 异步性

  进程以各自独立的、不可预知的速度向前推进。

- 结构特征

  程序段

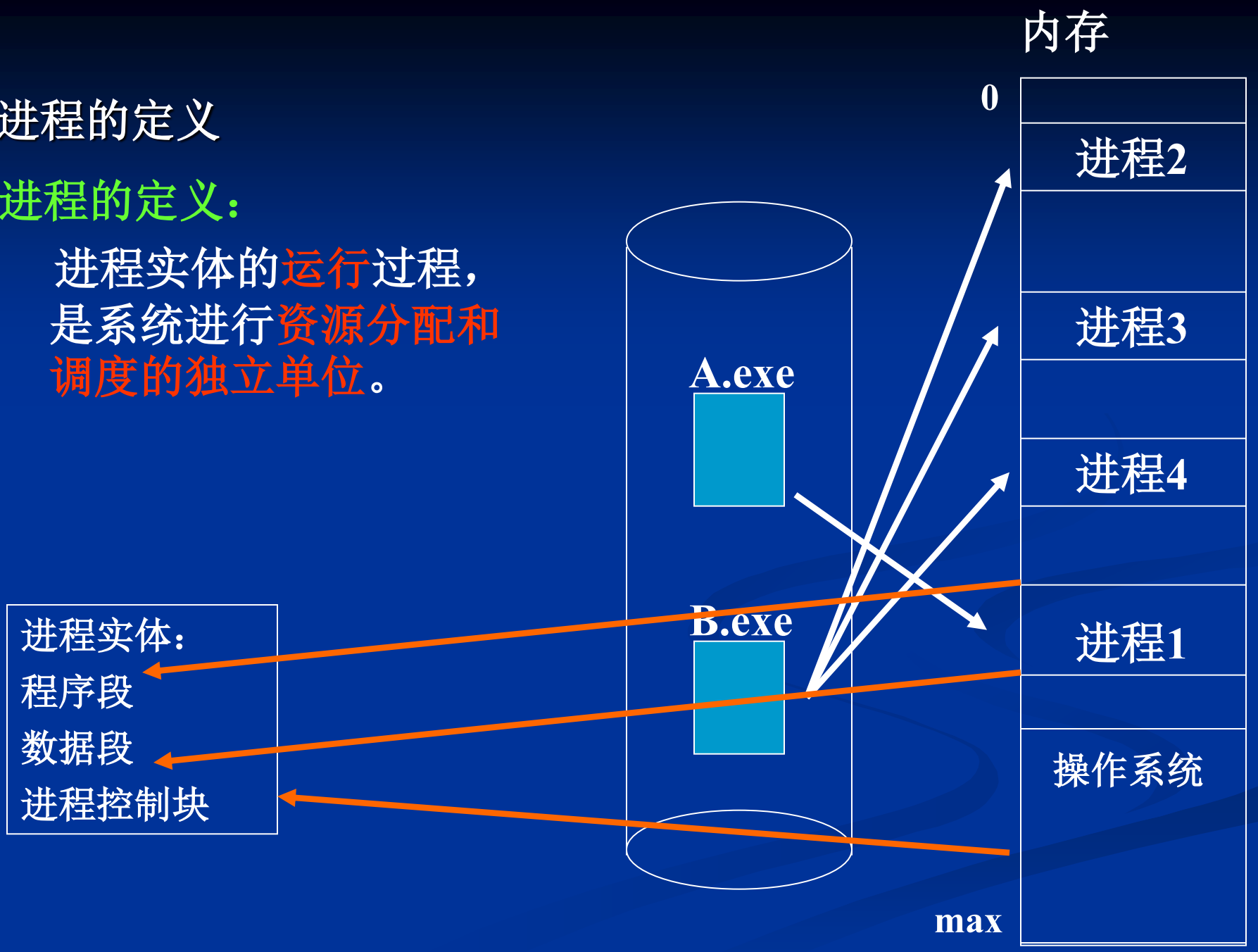
  数据段

  进程控制块（PCB）

(3) 进程的定义

进程的定义：

进程实体的运行过程，是系统进行资源分配和调度的独立单位。



(4) 进程的三种基本状态

- 进程基本状态

  1) 就绪状态

  进程已经获得除处理机之外的所有资源，一旦获得处理机就可以立即执行；

  2) 执行状态

  一个进程已获得了必要的资源，正在处理机上运行；

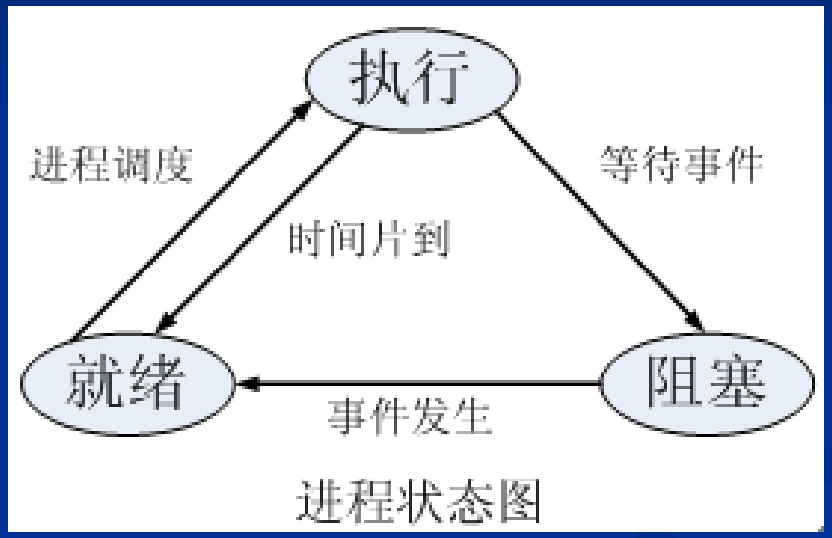
  3) 阻塞状态

  正在执行的进程，由于发生某事件而暂停无法继续执行，（如等待I/O、时间片到）

- 进程并非固定处于某种状态，其状态会随着自身的运行和和外界条件的变化而变化。

(4) 进程的三种基本状态

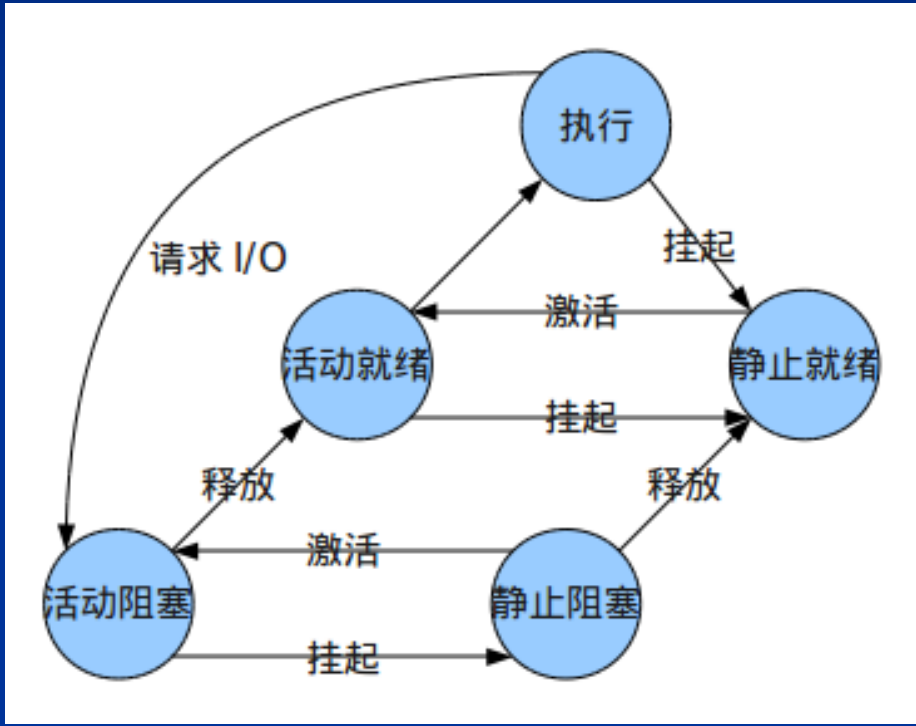
- 进程基本状态之间的转化



进程状态图

(4) 进程的三种基本状态

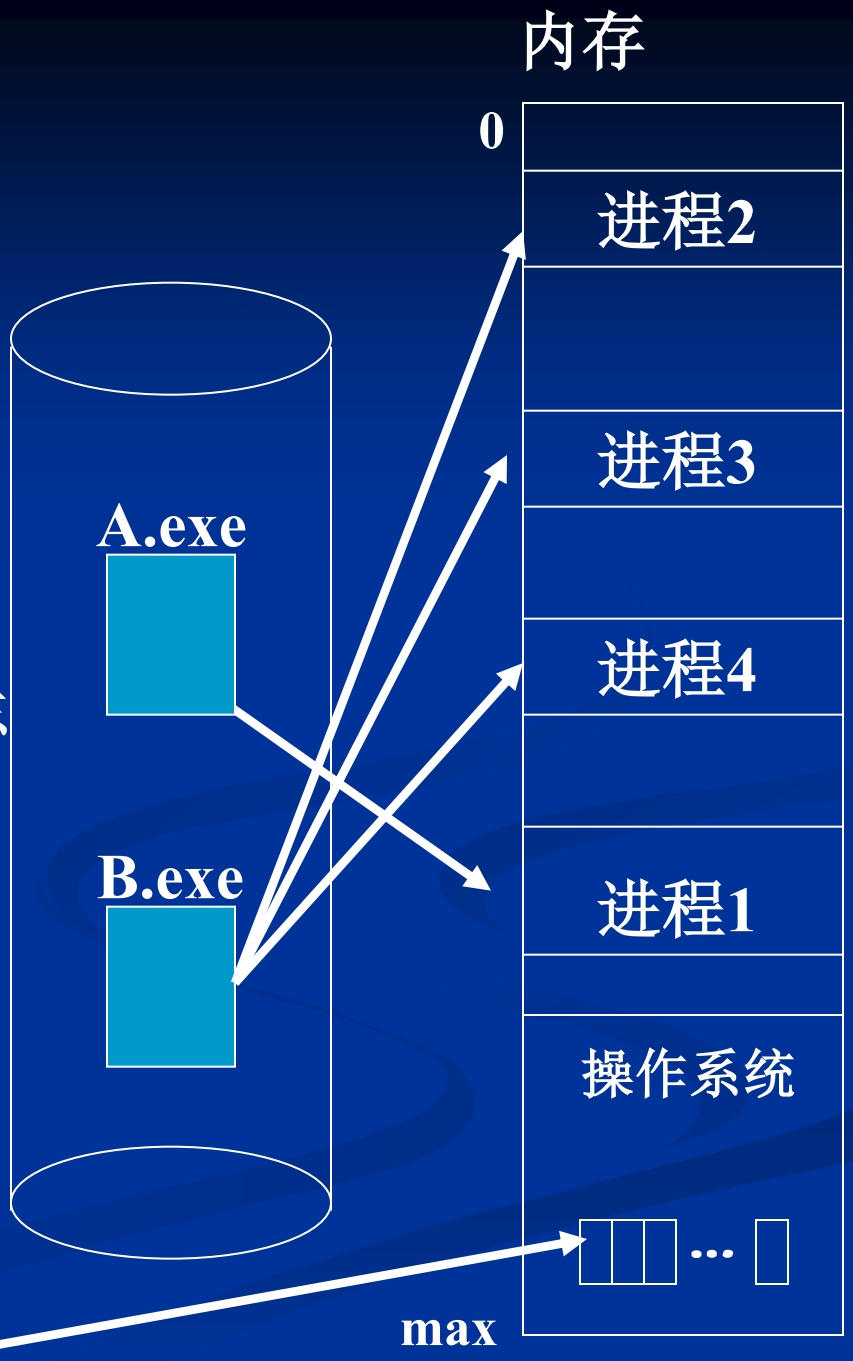
- 引入挂起状态的进程状态转化



2.1 进程的基本概念
2.1.5 进程控制块

(1) 进程控制块基本概念

- Process Control Block, PCB
- OS管理进程的数据结构
- 进程控制块是进程存在的唯一标志
- 进程控制块位置：操作系统内核



(2) 进程控制块的内容

- 进程标志符
- 处理机状态
- 进程调度信息
- 进程控制信息

进程名
当前状态
优先数
现场保留区
指示处于同一状态进程的链指针
资源清单
进程起始地址
家族关系
其他

（b）进程控制块的基本内容

```
/*
*********************************************************************************************************
*                                          TASK CONTROL BLOCK
*********************************************************************************************************
*/

typedef struct os_tcb {
    OS_STK        *OSTCBStkPtr;        /* Pointer to current top of stack                          */

#if OS_TASK_CREATE_EXT_EN
    void          *OSTCBExtPtr;        /* Pointer to user definable data for TCB extension         */
    OS_STK        *OSTCBStkBottom;     /* Pointer to bottom of stack                               */
    INT32U         OSTCBStkSize;       /* Size of task stack (in bytes)                            */
    INT16U         OSTCBOpt;           /* Task options as passed by OSTaskCreateExt()              */
    INT16U         OSTCBId;            /* Task ID (0..65535)                                       */
#endif

    struct os_tcb *OSTCBNext;          /* Pointer to next     TCB in the TCB list                  */
    struct os_tcb *OSTCBPrev;          /* Pointer to previous TCB in the TCB list                  */

#if (OS_Q_EN && (OS_MAX_QS >= 2)) || OS_MBOX_EN || OS_SEM_EN
    OS_EVENT      *OSTCBEventPtr;      /* Pointer to event control block                           */
#endif

#if (OS_Q_EN && (OS_MAX_QS >= 2)) || OS_MBOX_EN
    void          *OSTCBMsg;           /* Message received from OSMboxPost() or OSQPost()          */
#endif

    INT16U         OSTCBDly;           /* Nbr ticks to delay task or, timeout waiting for event    */
    INT8U          OSTCBStat;          /* Task status                                              */
    INT8U          OSTCBPrio;          /* Task priority (0 == highest, 63 == lowest)               */

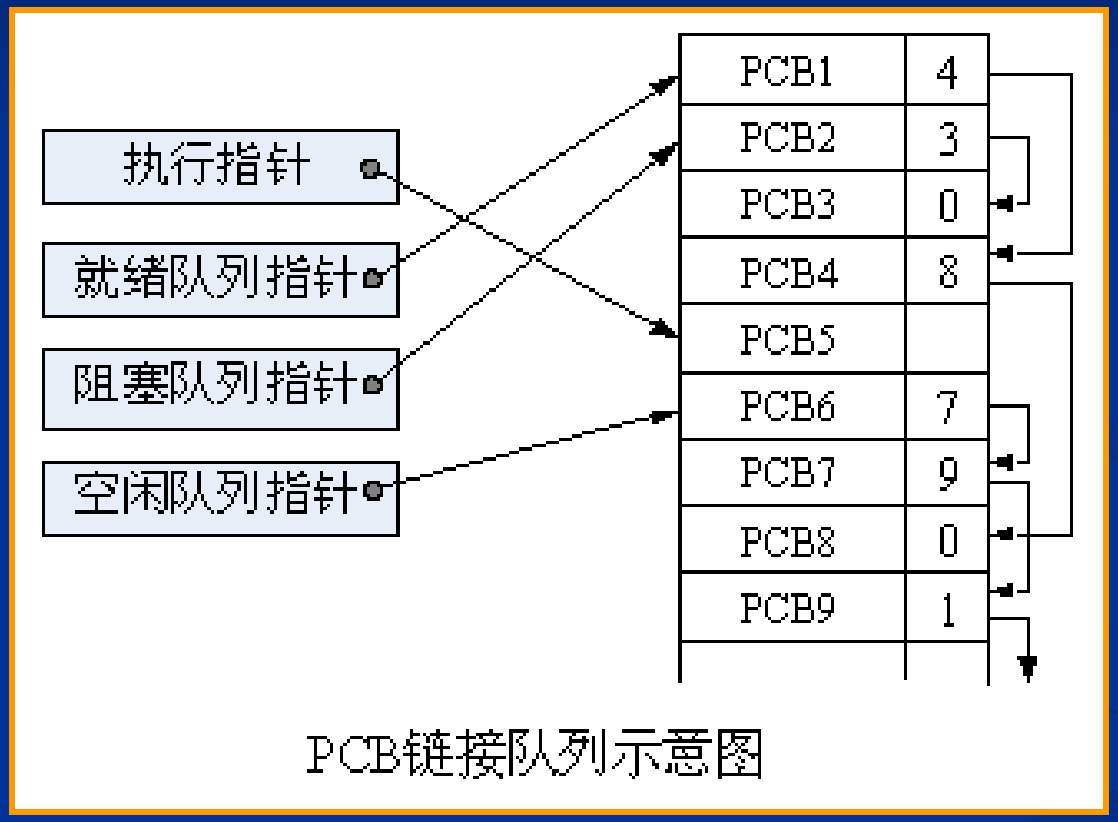
    INT8U          OSTCBX;             /* Bit position in group  corresponding to task priority (0..7) */
    INT8U          OSTCBY;             /* Index into ready table corresponding to task priority    */
    INT8U          OSTCBBitX;          /* Bit mask to access bit position in ready table           */
    INT8U          OSTCBBitY;          /* Bit mask to access bit position in ready group           */

#if OS_TASK_DEL_EN
    BOOLEAN        OSTCBDelReq;        /* Indicates whether a task needs to delete itself          */
#endif
} OS_TCB;

/*$PAGE*/
/*
*********************************************************************************************************
*                                          GLOBAL VARIABLES
*********************************************************************************************************
```

(3) 进程控制块的组织形式

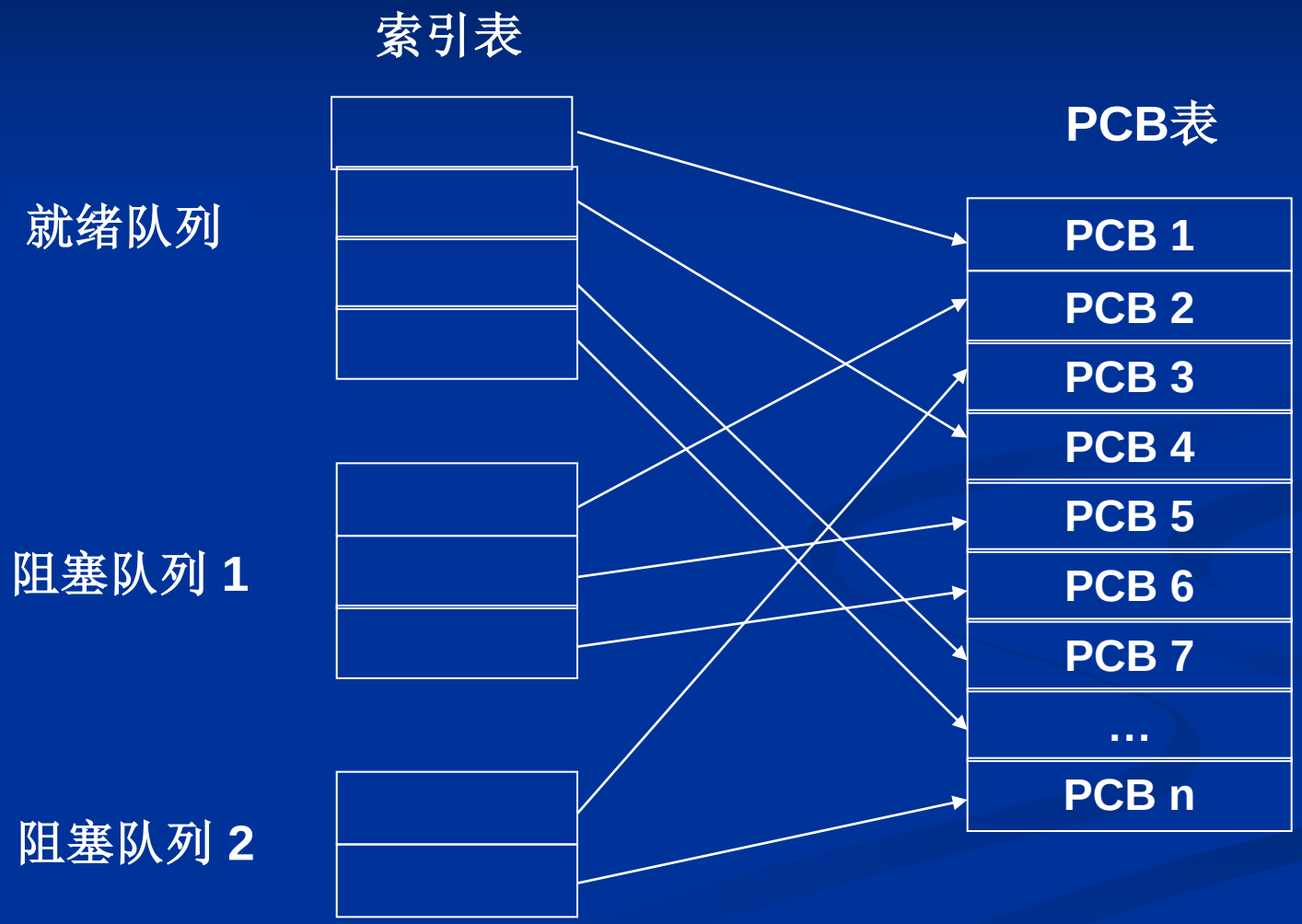
- 链接方式



PCB链接队列示意图

(3) 进程控制块的组织形式

- 索引方式



2.2 进程控制

2.2.1 进程的创建和撤消

(1) 进程的创建

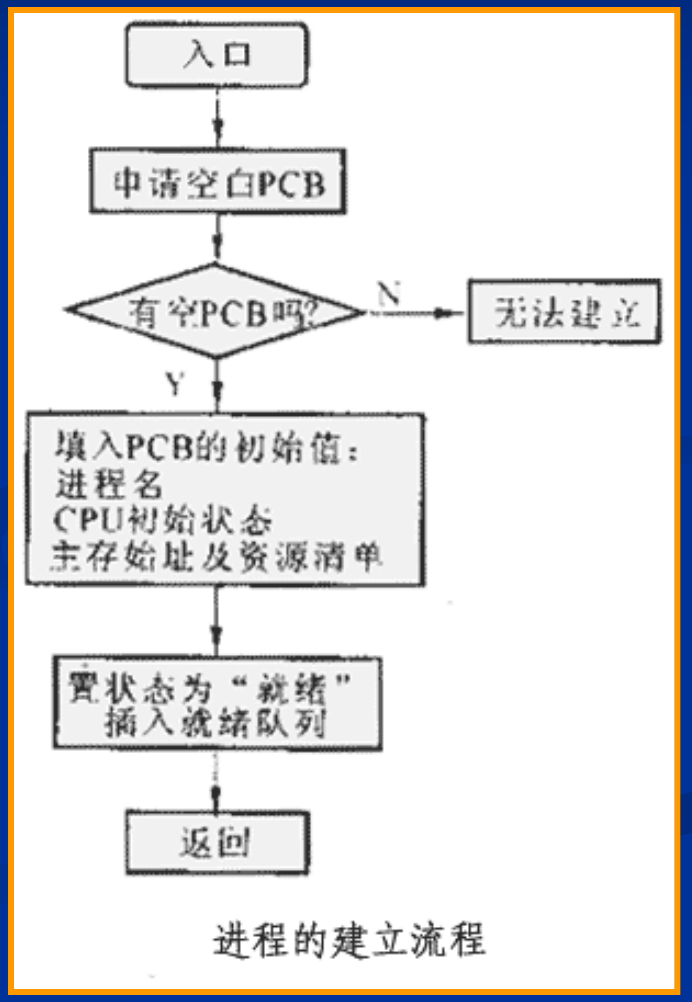
- 引起进程创建的事件
  - 用户登录
  - 作业调度
  - 提供服务

    例如：提供打印服务
  - 应用请求

    例如：程序自行创建
- 进程创建原语

  create()

- 创建进程的流程:

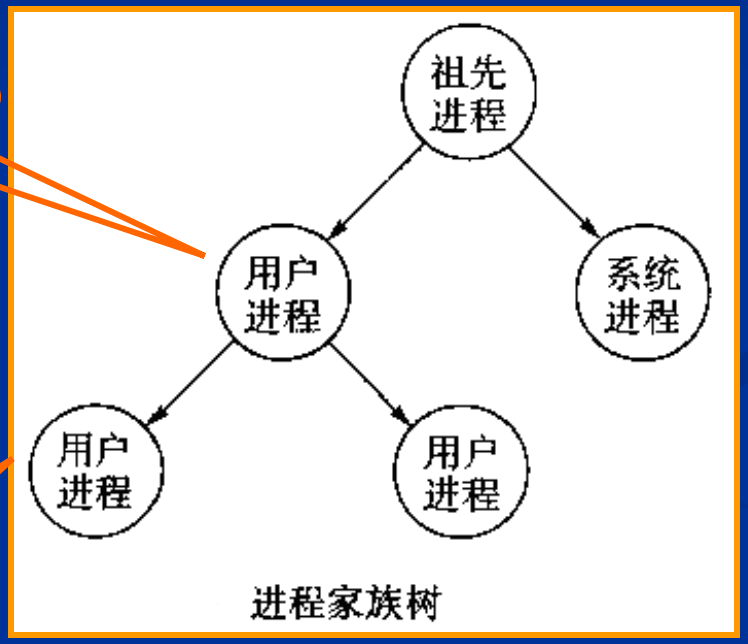


进程的建立流程

(1) 进程的创建

- 进程图(树）：

  体现进程的创建关系。



父进程

子进程

(2) 进程的撤消

- 引起进程撤消的事件
  - 正常结束
  - 异常结束
  - 外界干预

- 进程撤消原语

  exit()

- 撤消进程的流程:

  1) 根据被终止进程的标识符，从PCB集合中检索出该进程的PCB，从中读出该进程的状态。

  2) 若被终止进程正处于执行状态，应立即终止该进程的执行，并置调度标志为真，用于指示该进程被终止后应重新进行调度。

  3) 若该进程还有子孙进程，还应将其所有子孙进程予以终止，以防他们成为不可控的进程。

  4) 将被终止进程所拥有的全部资源，或者归还给其父进程，或者归还给系统。

  5) 将被终止进程（它的PCB）从所在队列（或链表）中移出，等待其他程序来搜集信息。

2.2 进程控制
2.2.2 进程的阻塞与唤醒

(1) 进程的阻塞

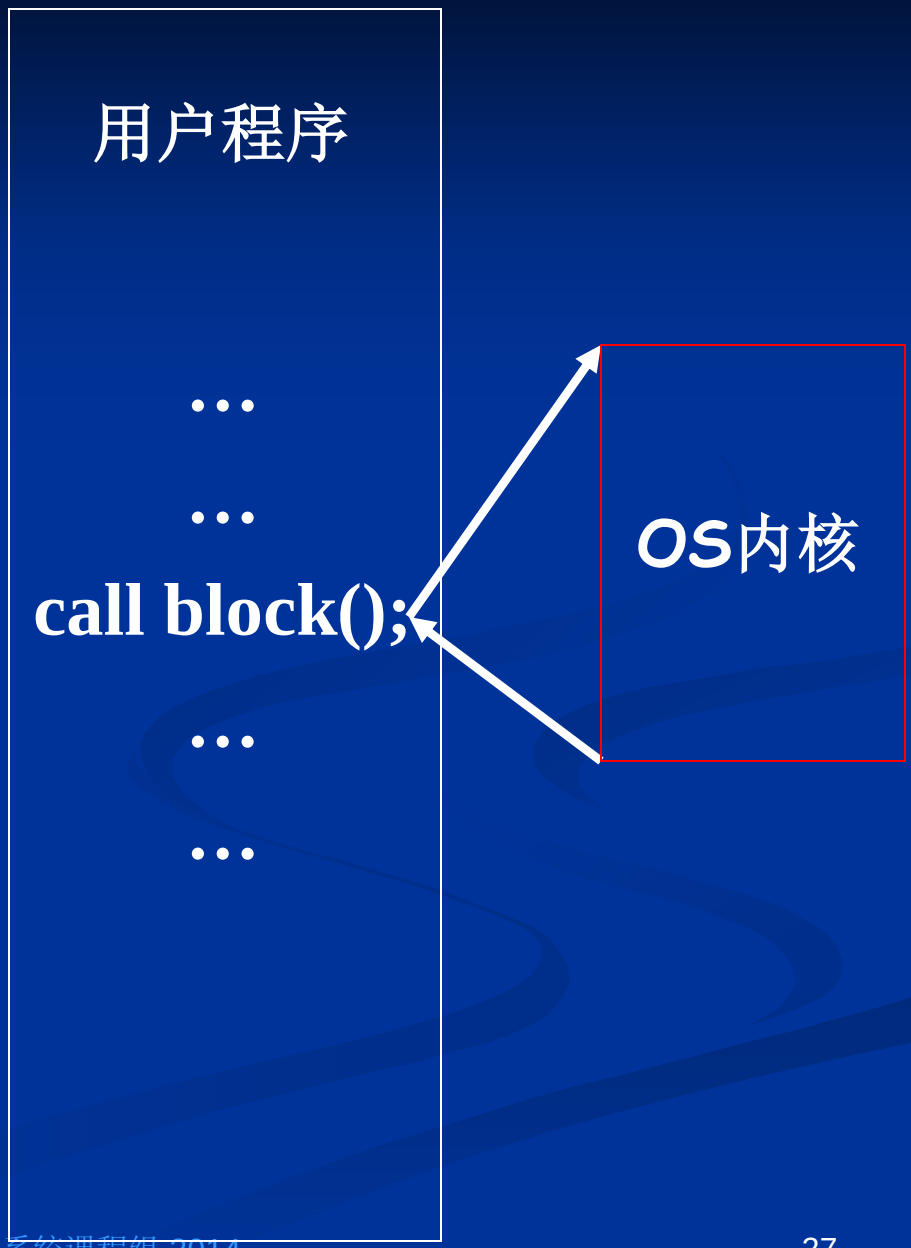
- 引起进程阻塞的事件
  - 请求系统服务
  - 请求操作
  - 等待数据、资源
  - 等待工作任务

- 进程阻塞原语

  block()

  进程自己阻塞自己

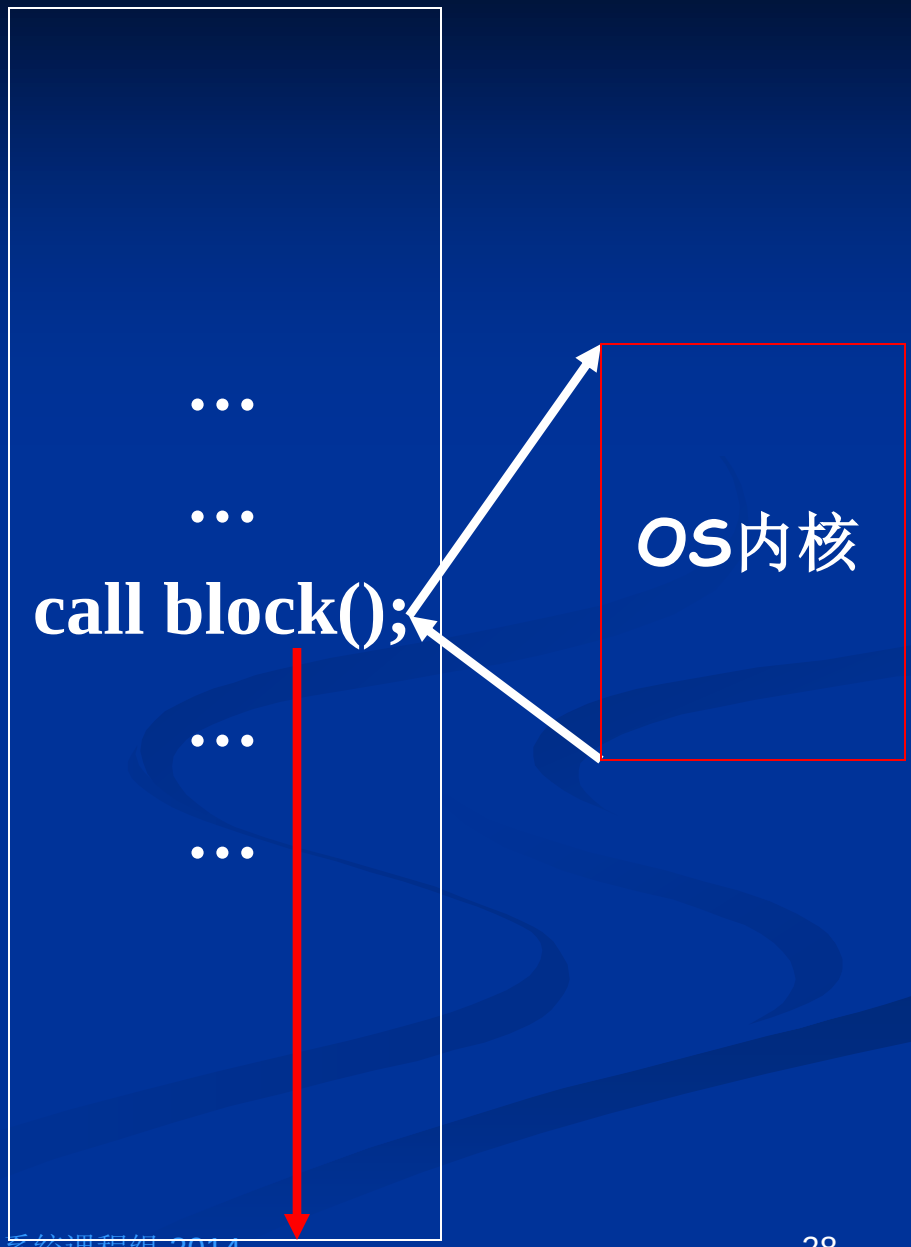
- 阻塞原理



(2) 进程的唤醒

- 引起进程唤醒的事件
  - 允许系统服务
  - 允许操作
  - 数据、资源到达
  - 工作任务到达
- 进程唤醒原语

  wakeup()



别的进程唤醒

2.2 进程控制
2.2.2 进程的挂起与激活

(1) 进程的挂起

- 引起进程挂起的事件
  - 调试系统
  - 暂停程序执行
- 进程挂起原语

  suspend()

(2) 进程的激活

- 引起进程激活的事件
  - 调试系统结束
  - 程序继续执行
- 进程激活原语

  activate()

2.3 进程同步
2.3.1 进程同步的基本概念

(1) 什么是进程同步

- 进程同步问题的产生

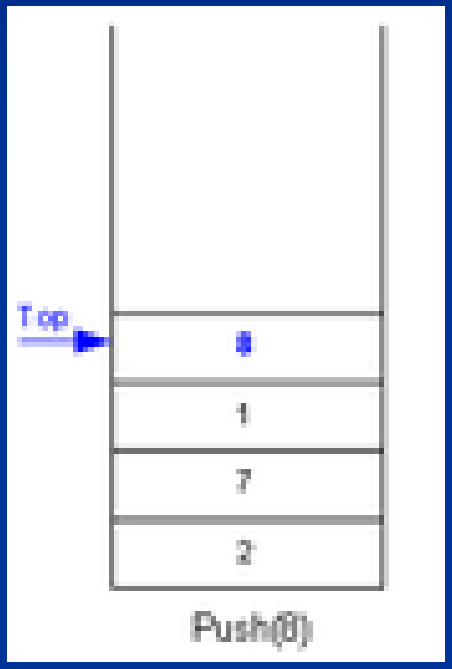
  在多道程序的环境中，系统中的多个进程可以并发执行，同时它们又要共享系统中的资源，这些资源有些是可共享使用的，如磁盘，有些是以独占方式使用的，如打印机。由此将会引起一系列的矛盾，产生错综复杂的相互制约的关系。

- 两种形式的制约关系：
  - 间接相互制约关系：共享某种系统资源。
  - 直接相互制约关系：主要源于进程间合作。

2.3 进程同步

(2) 临界资源

- 入栈操作




```
Parbegin
  VAR  top : integer = -1 ;
       stack: array[0..n-1] of item;
  Program A :
  Begin
     while ( TRUE )
        top : = (top+1) mod n;
        stack[top] := x ;
     end while
  end
  Program B :
  Begin
     while ( TRUE )
        top : = (top+1) mod n;
        stack[top] := y ;
     end while
  End
Parend
```

2.3 进程同步

(2) 临界资源

- **互斥共享的资源：临界资源**

  虽然计算机系统中多个进程可以共享系统中的各种资源，然而有些资源一次只能为一个进程使用。一次仅能为一个进程所使用的资源称为临界资源。

- **临界资源例子**

  硬件资源：打印机；

  软件资源：内存变量、指针、数组。

如何实现临界资源的互斥访问？

临界资源

```
Parbegin
  VAR  top : integer = -1 ;
       stack: array[0..n-1] of item;
  Program A :
  Begin
     while ( TRUE )
       top : = (top+1) mod n;
       stack[top] := x ;
     end while
  end
  Program B :
  Begin
     while ( TRUE )
       top : = (top+1) mod n;
       stack[top] := y ;
     end while
  End
Parend
```

2.3 进程同步

(3) 临界区

- **临界区的定义**

  进程中访问临界资源的代码段称为临界区。

- **策略**

  临界资源访问互斥

  ↓

  临界区访问互斥

```
Parbegin
  VAR  top : integer = -1 ;
       stack: array[0..n-1] of item;
  Program A :
  Begin
     while ( TRUE )
        top : = (top+1) mod n;
        stack[top] := x ;
     end while
  end
  Program B :
  Begin
     while ( TRUE )
        top : = (top+1) mod n;
        stack[top] := y ;
     end while
  End
Parend
```

临界资源

临界区

临界区

2.3 进程同步

(4) 临界区访问控制模型

```
repeat
        critical section ;
        remainder section ;
until false
```



```
repeat
    entry section;
        critical section ;
    exit section ;
        remainder section ;
until false
```

进入区

退出区

(5) 同步机制遵循的原则

- **空闲让进：** 当没有进程处于临界区时，请求进入临界区的进程立即进入
- **忙则等待：** 当已有进程进入临界区时，其他试图进入临界区进程须等待
- **有限等待：** 对要求访问临界资源进程，保证能在有限时间内进入临界区
- **让权等待：** 当进程不能进入临界区时，应释放处理机。

2.3 进程同步
2.3.2 实现进程同步的早期方

(1) 软件解法1：按需访问

- 违背什么原则？

  忙则等待

资源空闲标志

```
VAR free:boolean := false ;
Program P1:
Begin
  repeat
    while( free ) ;
    free := true ;
      critical section ;
    free := false ;
      remainder section ;
  until false
End

Program P2:
Begin
  repeat
    while( free ) ;
    free := true ;
      critical section ;
    free := false ;
      remainder section ;
  until false
End
```

2.3 进程同步
2.3.2 实现进程同步的早期方

(2) 软件解法2：轮询

轮流标志

```
VAR turn:integer := 1 ;
Program P1:
Begin
  repeat
    while( turn = 2) ;

       critical section ;
    turn := 2 ;
       remainder section ;
  until false
End

Program P2:
Begin
  repeat
    while( turn = 1 ) ;

       critical section ;
    turn := 1 ;
       remainder section ;
  until false
End
```

- 违背什么原则？

  严格限制资源访问顺序

2.3 进程同步
2.3.2 实现进程同步的早期方

(3) 软件解法3：访前先看

```
VAR pturn,qturn:boolean := false,false ;
Program P:
Begin
  repeat
    pturn := true ;
    while( qturn ) ;
        critical section ;
    pturn := false ;
        remainder section ;
  until false
End

Program Q:
Begin
  repeat
    qturn := true ;
    while( pturn ) ;
        critical section ;
    qturn := false ;
        remainder section ;
  until false
End
```

- 违背什么原则？

  空闲让进

2.3 进程同步
2.3.2 实现进程同步的早期方法

(4) 软件解法4：Peterson算法,1981

```
#define  FALSE  0
#define  TRUE   1
#define   N        2                          // 进程的个数
int  turn;                          // 轮到谁?
int  interested[N];                 // 兴趣数组，初始值均为FALSE

void  enter_region ( int  process)// process = 0 或 1
{
     int  other;                    // 另外一个进程的进程号
     other  =  1  -  process;
     interested[process]  =  TRUE;// 表明本进程感兴趣
     turn  =  process;              // 设置标志位
     while( turn == process && interested[other] == TRUE);
}

void  leave_region ( int  process)
{
    interested[process]  =  FALSE;    // 本进程已离开临界区
}
```

```
Program P1:
Begin
  repeat
    enter_region(0 );
        critical section ;
    leave_region(0);
        remainder section ;
  until false
End

Program P2:
Begin
  repeat
    enter_region(1 );
        critical section ;
    leave_region(1);
        remainder section ;
  until false
End
```

2.3 进程同步
2.3.2 实现进程同步的早期方法

(5) 硬件解法1：TSL指令

```
// 复制锁lock的值到register，并且置lock为1
function TSL(register, lock  )
begin
    register := lock ;
    lock := 1 ;
end
```

```
;;; 定义两个过程enter_region 和leave_region
enter_region :
   TSL register, lock    ; 复制锁lock的值到register，并且置
   CMP register, #0
   JNZ enter_region
   RET

leave_region:
   MOVE lock, #0
   RET
```

```
VAR lock : integer := 0 ;
Program P1:
Begin
  repeat
    enter_region(lock );
       critical section ;
    leave_region(lock);
       remainder section ;
  until false
End

Program P2:
Begin
  repeat
    enter_region(lock);
       critical section ;
    leave_region(lock);
       remainder section ;
  until false
End
```

2.3 进程同步

2.3.2 实现进程同步的早期方法

(6) 硬件解法2：SWAP指令

```
// 交换锁lock和key的值
function SWAP(lock, key  ) {
    var tmp : boolean := lock ;
    lock := key ;
    key := tmp ;
}
```

```
function enter_region( var lock : boolean )
Var key:boolean ;  //局部变量
Begin
   key := true;
   while(key)
      SWAP(lock,key);
end

function leave_region( var lock : boolean )
Begin
    lock := false;
end
```

```
VAR lock :boolean := false ;
Program P1:
Begin
   repeat
    enter_region(lock );
        critical section ;
    leave_region(lock);
        remainder section ;
   until false
End

Program P2:
Begin
   repeat
    enter_region(lock);
        critical section ;
    leave_region(lock);
        remainder section ;
   until false
End
```

2.3 进程同步
2.3.2 实现进程同步的早期方法

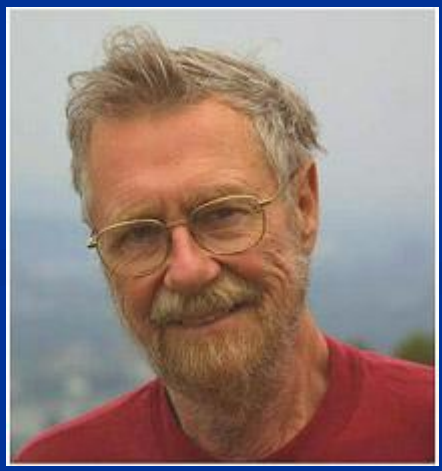
(7) 小结

- 软件解法
  - 忙等待
  - 实现过于复杂，需要高的编程技巧

- 硬件解法
  - 简单、有效，特别适用于多处理机
  - 缺点：忙等待

3.7.2 银行家算法

- Dijkstra
  荷兰计算机科学家Edsger Wybe Dijkstra;
  1972年ACM 图灵奖获得者
- 贡献:
  提出“goto有害论”;
  提出信号量和PV原语;
  解决了有趣的“哲学家聚餐”问题;
  最短路径算法(SPF)的创造者;
  第一个Algol 60编译器的设计者和实现者;
  THE操作系统的设计者和开发者;



2.3 进程同步
2.3.3 信号量机制

(1) 整数型信号量，Dijkstra,1965

- 定义
  - 整数信号量S;
  - 两个原子操作：wait/signal(P/V)

```
function Wait ( var S: integer )
Begin
   while ( S <= 0 ) do no_op();
   S := S-1;
end

function Signal( var S : integer )
Begin
    S := S+1;
end
```

```
VAR S:integer := 1 ;
Program P1:
Begin
  repeat
   wait(S);
       critical section ;
   signal(S);
       remainder section ;
  until false
End

Program P2:
Begin
  repeat
   wait(S);
       critical section ;
   signal(S);
       remainder section ;
  until false
End
```

2.3 进程同步
(2) 记录型信号量

- 定义
  - 记录型信号量S;
  - 两个原子操作：wait/signal(P/V)

```
type semaphore = record
    value: integer ;                    // 资源数量
    L: list of process ;                // 阻塞进程队列
end

function wait ( var S ： Semaphore )
Begin
   S.value := S.value-1;
   if ( S.value < 0 ) then block (S.L);
end
function signal(var S ： Semaphore )
Begin
   S.value := S.value+1;
   if ( S.value <= 0 ) then wakeup (S.L);
end
```

```
VAR S:semaphore:= 1 ;
Program P1:
Begin
  repeat
   wait(S );
       critical section ;
   signal(S);
       remainder section ;
  until false
End

Program P2:
Begin
  repeat
   wait(S);
       critical section ;
   signal(S);
       remainder section ;
  until false
End
```

2.3 进程同步

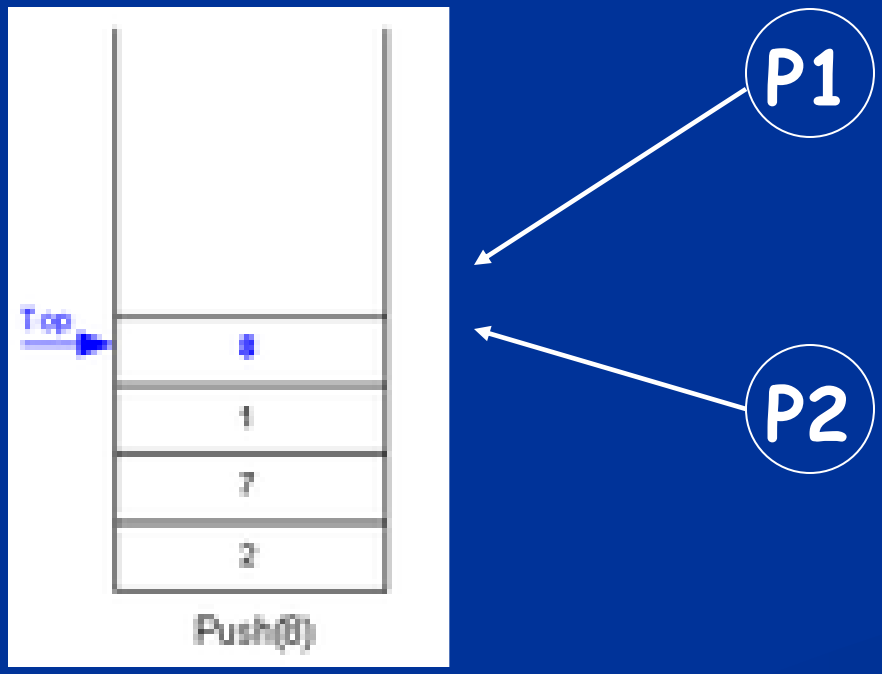
(2) 记录型信号量

- 信号量S的物理含义
  - 对信号量的每次wait操作，意味着进程请求一个单位的该类资源，因此描述为S.value:= S.value-1；当S.value＜0时，表示该类资源已分配完毕，因此进程应进行自我阻塞，放弃处理机，并插入到信号量链表S.L中。
  - 对信号量的每次signal操作，表示执行进程释放一个单位资源，故S.value:= s.value+1操作表示资源数目加1。若加1后仍是S.value＜=0，则表示在该信号量链表中，仍有等待该资源的进程被阻塞，将S.L链表中的第一个等待进程唤醒。
  - S. value:
    - S. value>=0: 资源数量;
    - S. value<0 : |S. value|为阻塞进程的数量;

2.3 进程同步
2.3.4 信号量的应用

(1) 信号量用于互斥

例1：2个进程进行堆栈的入栈




```
Parbegin
  VAR  top : integer = -1 ;
       stack: array[0..n-1] of item;
       mutex: semaphore :=1 ;
  Process P1 :
  Begin
     while ( TRUE )
       wait( mutex ) ;
       top : = (top+1) mod n;
       stack[top] := x ;
       signal(mutex ) ;
     end while
  end
  Process P2 :
  Begin
     while ( TRUE )
       wait( mutex ) ;
       top : = (top+1) mod n;
       stack[top] := y ;
       signal(mutex ) ;
     end while
  End
Parend
```

2.3 进程同步

(1) 信号量用于互斥

例2：过独木桥问题，一次只能过1人.





河水　　河水



```
Parbegin
  Process S2N :
  Begin
     repeat
        go across the bridge ;
     until false
  end
  Process N2S :
  Begin
     repeat
        go across the bridge ;
     until false
  End
Parend
```

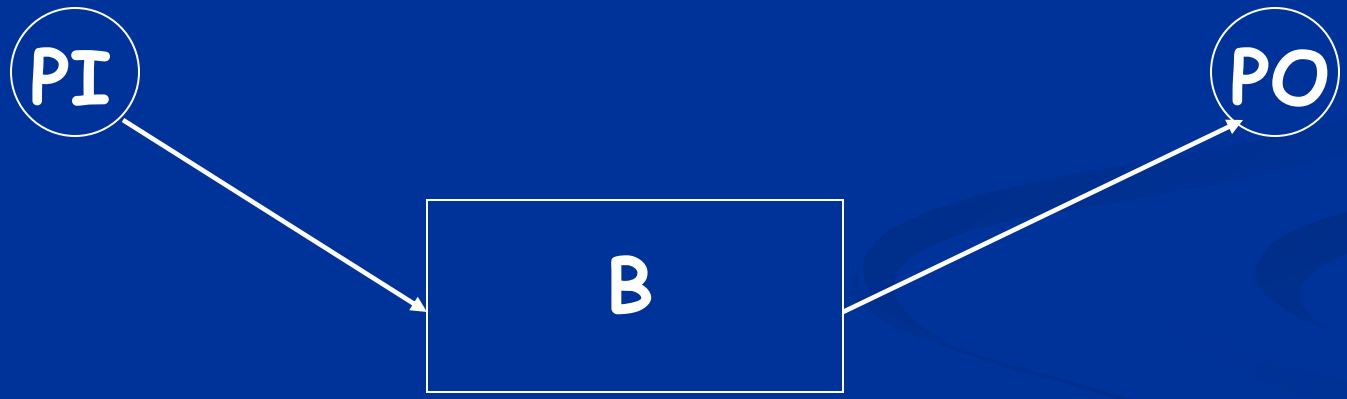


```
Parbegin
  VAR  mutex : semaphore = 1 ;
  Process S2N :
  Begin
     repeat
        wait( mutex ) ;
        go across the bridge ;
        signal(mutex ) ;
     until false
  end
  Process N2S :
  Begin
     repeat
        wait( mutex ) ;
        go across the bridge ;
        signal(mutex ) ;
     until false
  End
Parend
```

2.3 进程同步

(2) 信号量用于同步

例3：PI,PO两进程共享缓冲区B,PI负责读入数据并保存在B中，PO负责打印B中数据，要求每个数据必须且只打印一次，不许漏打或重复打印。



```
Parbegin
  Process PI :
  Begin
     repeat
        read x from I/O ;
        B := x ;
     until false
  end
  Process PO :
  Begin
     repeat
        y := B ;
        print( y) ;
     until false
  End
Parend
```

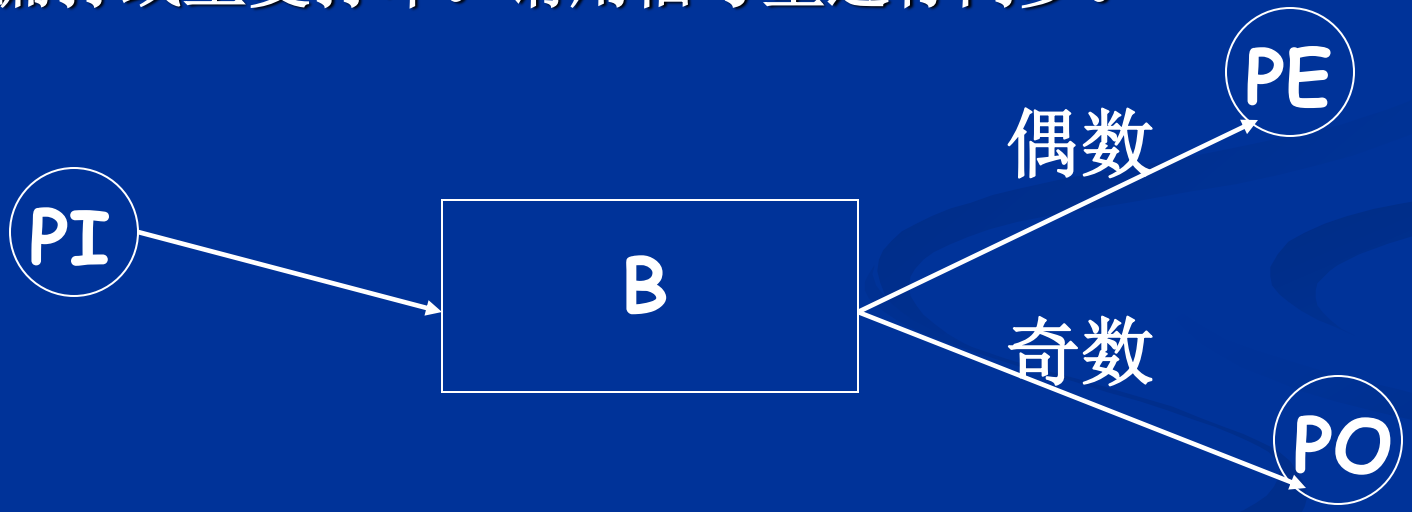


```
Parbegin
  VAR  S1,S2 : semaphore = 1, 0 ;
  Process PI :
  Begin
     repeat
        read x from I/O ;
        wait( S1 ) ;
        B := x ;
        signal(S2 ) ;
     until false
  end
  Process PO :
  Begin
     repeat
        wait( S2 ) ;
        y := B ;
        signal(S1 ) ;
        print( y) ;
     until false
  End
Parend
```

2.3 进程同步

(2) 信号量用于同步

例3：PI,PE,PO三进程通过共享缓冲区B协作,PI负责产生随机数并保存在B中，当B中数据为奇数时由PO负责打印，当B中数据为偶数时由PE负责打印，要求每个数据必须且只打印一次，不许漏打或重复打印。请用信号量进行同步。



```
Parbegin
  Process PIN :
  Begin
    repeat
      generate x randomly;

      B := x ;

    until false
  end
  Process PE :
  Begin
    repeat

      y := B ;

      print( y) ;
    until false
  End
Process PO :
  Begin
    repeat

      z := B ;

      print( z) ;
    until false
  End
Parend
```

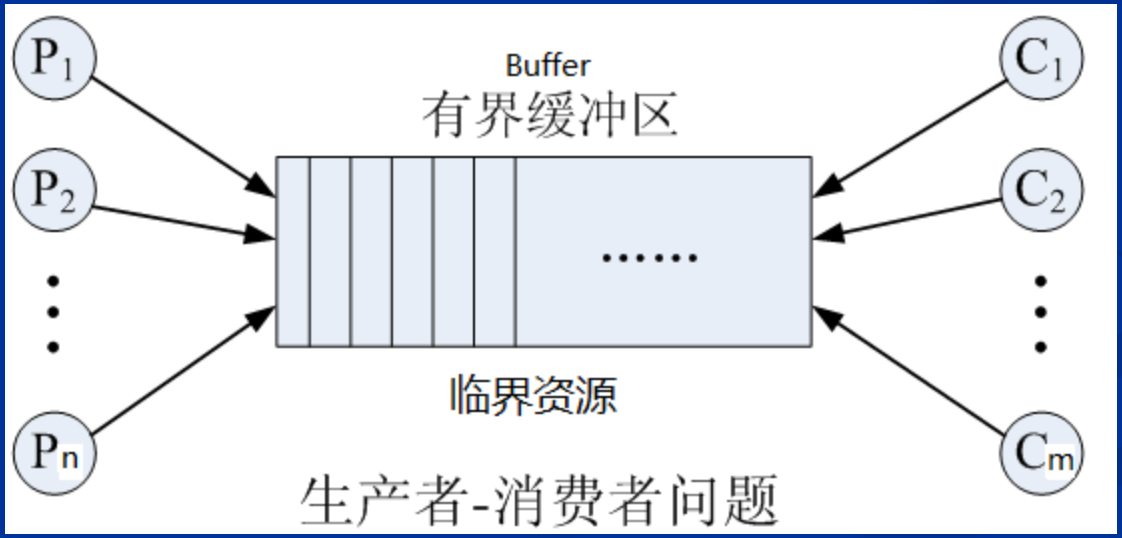


```
Parbegin
  var S,SO,SE:semaphore :=1,0,0;
  Process PIN :
  Begin
    repeat
      generate x randomly;
      wait(S);
      B := x ;
      if ( x is odd ) signal( SO );
        else          signal( SE ) ;
    until false
  end
  Process PE :
  Begin
    repeat
      wait(SE);
      y := B ;
      signal( S );
      print( y) ;
    until false
  End
Process PO :
  Begin
    repeat
      wait(SO);
      z := B ;
      signal( S );
      print( z) ;
    until false
  End
Parend
```

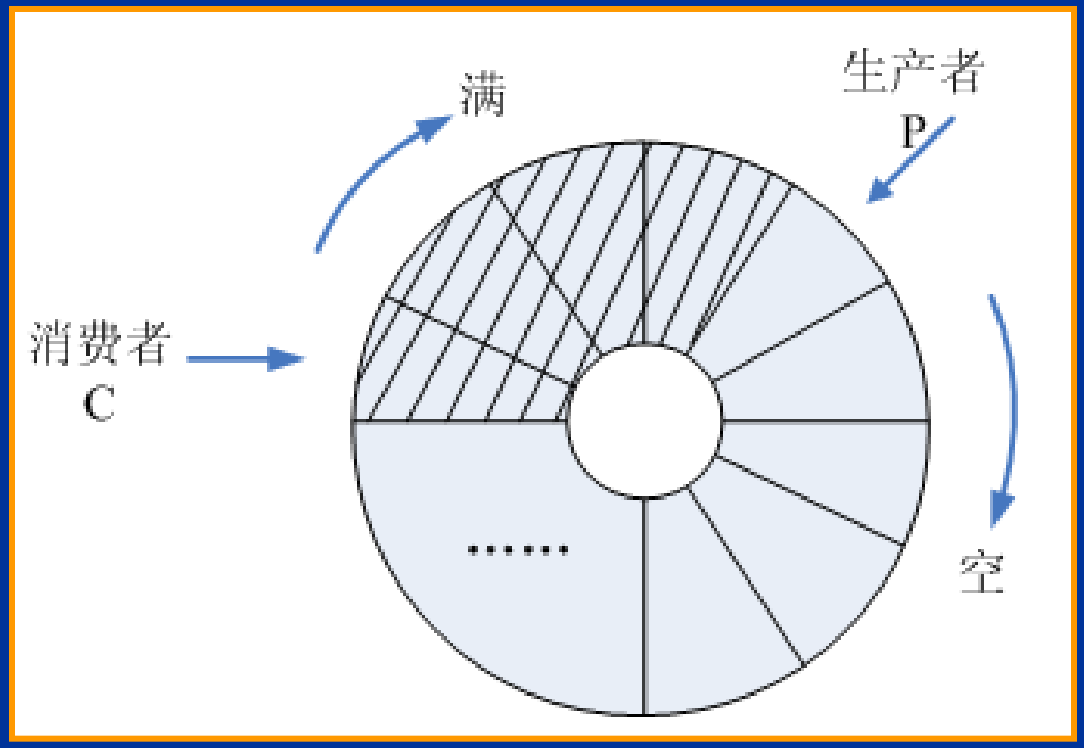
2.4 经典进程同步问题
2.4.1 生产者-消费者问题

(1) 生产者消费者问题

多个生产者、多个消费者通过共享含n个缓冲区的缓冲池Buffer协作,其中生产者负责生产数据并投入缓冲池，消费者从缓冲池中取数据消费，生产者和消费者，每次生产/消费1个数据，要求每个数据必须且只被消费一次。请用信号量进行同步。

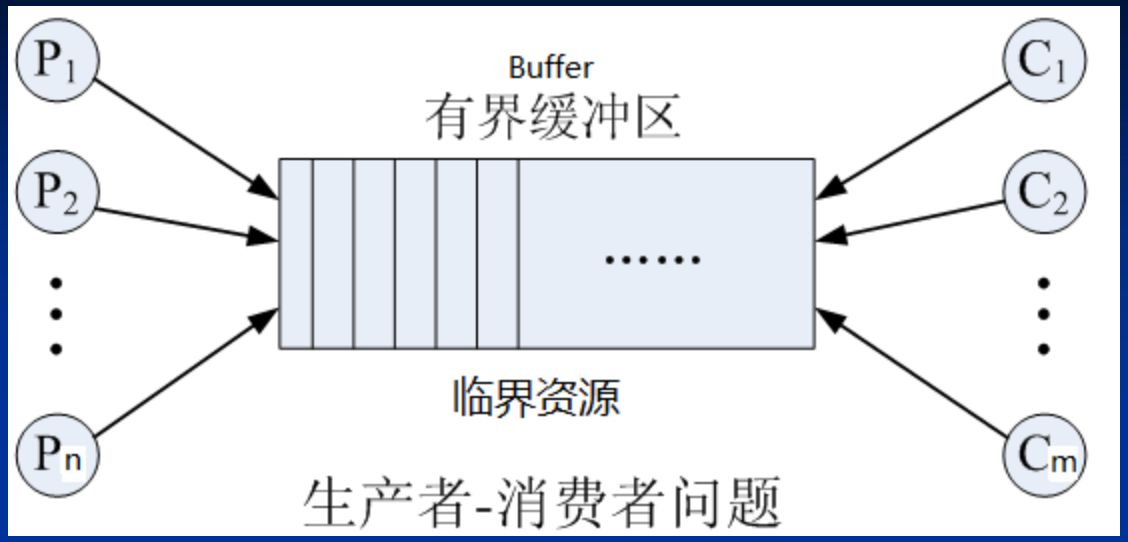


- 有界缓冲池 --> 循环缓冲区



- 循环队列
  - 队列数据结构
  - 入队操作
  - 出队操作

进程资源共享关系和同步关系分析



- 生产者进程：空缓冲区资源
- 消费者进程：满缓冲区资源

```
VAR
      in,out: integer := 0, 0 ;
      Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
        repeat
           produce an item in nextp ;

           Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;

       until flase
   end

Consumer:
begin
     repeat

        nextc = Buffer(out);
        out := (out + 1 ) mod n ;

        consume the item nextc ;
     until flase
end
Parend
```

数据结构

入队

出队

```
VAR empty, full: semaphore :=  n, 0 ;
      in,out: integer := 0, 0 ;
      Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
        repeat
           produce an item in nextp ;
           wait( empty ) ;

           Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;

           signal( full ) ;
       until flase
   end

Consumer:
begin
     repeat
        wait( full ) ;

        nextc = Buffer(out);
        out := (out + 1 ) mod n ;

        signal( empty ) ;
        consume the item nextc ;
     until flase
end
Parend
```

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
      in,out: integer := 0, 0 ;
      Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
        repeat
           produce an item in nextp ;
           wait( empty ) ;
           wait( mutex ) ;
           Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;
           signal( mutex ) ;
           signal( full ) ;
        until flase
   end

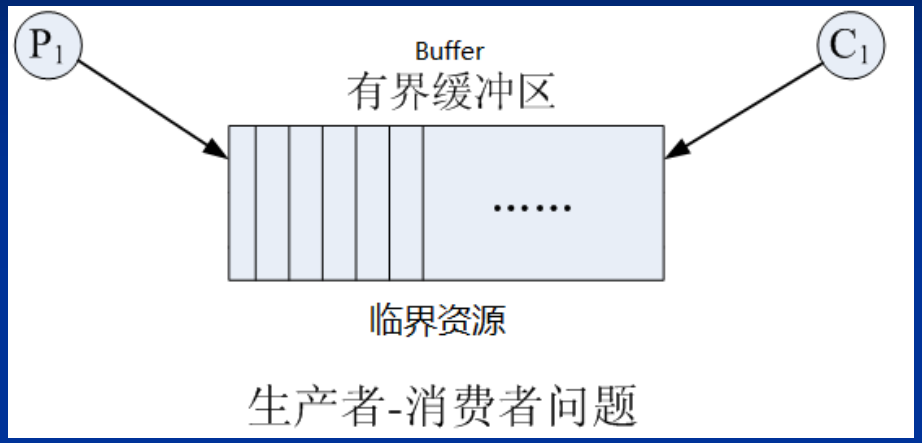
Consumer:
begin
     repeat
        wait( full ) ;
        wait( mutex ) ;
        nextc = Buffer(out);
        out := (out + 1 ) mod n ;
        signal( mutex ) ;
        signal( empty ) ;
        consume the item nextc ;
     until flase
end
Parend
```

(2) 问题思考

- 单生产者单消费者、多缓冲时
- 多生产者多消费者、单缓冲时
- 单生产者单消费者、单缓冲时
- 允许生产者写时，消费者可读
- 当缓冲区无限大；
- 每个消息都要每个消费者消费1次
- 调整生产者wait顺序；
- 调整消费者wait顺序；
- 调整signal顺序；

(2) 问题思考

- 单生产者单消费者、多缓冲时

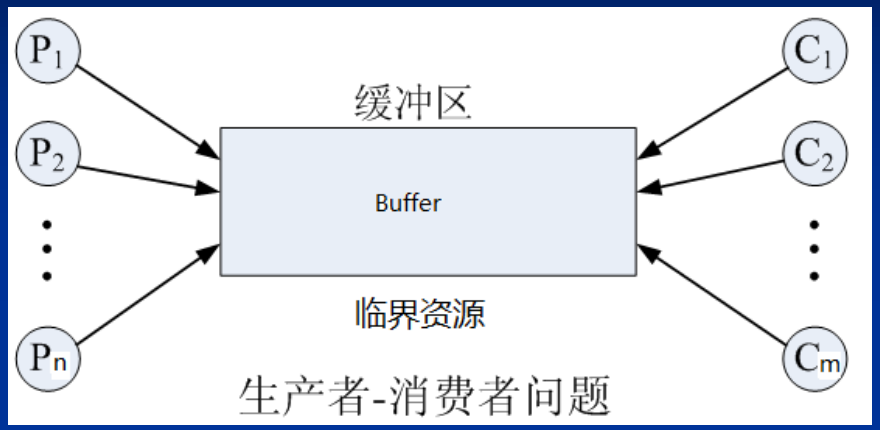


生产者-消费者标准程序

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
 Producer:
 begin
    repeat
         produce an item in nextp ;
         wait( empty ) ;
         wait( mutex ) ;
         Buffer(in) := nextp ;
          in := (in + 1 ) mod n ;
          signal( mutex ) ;
          signal( full ) ;
    until flase
 end
 Consumer:
 begin
     repeat
         wait( full ) ;
         wait( mutex ) ;
         nextc = Buffer(out);
         out := (out + 1 ) mod n ;
         signal( mutex ) ;
         signal( empty ) ;
         consume the item nextc ;
     until flase
 end
Parend
```

(2) 问题思考

- 多生产者多消费者、单缓冲时

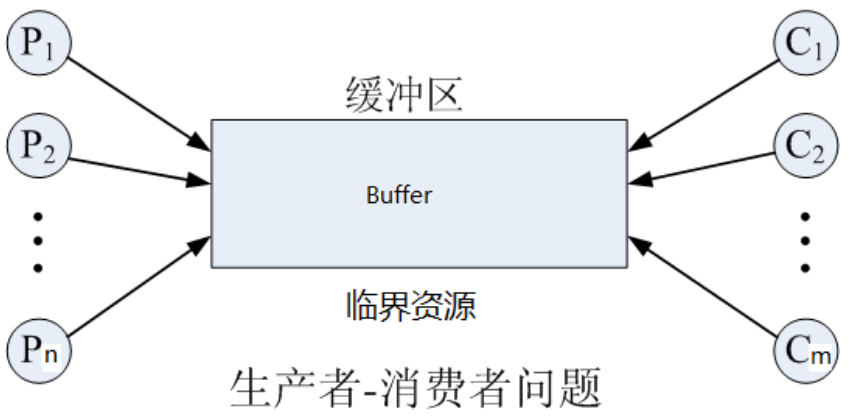


生产者–消费者标准程序

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
      in,out: integer := 0, 0 ;
      Buffer: array [0..n-1] of item ;
Parbegin
 Producer:
 begin
    repeat
       produce an item in nextp ;
       wait( empty ) ;
       wait( mutex ) ;
       Buffer(in) := nextp ;
        in := (in + 1 ) mod n ;
        signal( mutex ) ;
        signal( full ) ;
    until flase
 end
 Consumer:
 begin
     repeat
       wait( full ) ;
       wait( mutex ) ;
       nextc = Buffer(out);
       out := (out + 1 ) mod n ;
       signal( mutex ) ;
       signal( empty ) ;
       consume the item nextc ;
     until flase
 end
Parend
```

(2) 问题思考

- 多生产者多消费者、单缓冲时




```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
          repeat
             produce an item in nextp ;
             wait( empty ) ;

             Buffer(in) := nextp ;
             in := (in + 1 ) mod n ;

             signal( full ) ;
          until flase
   end

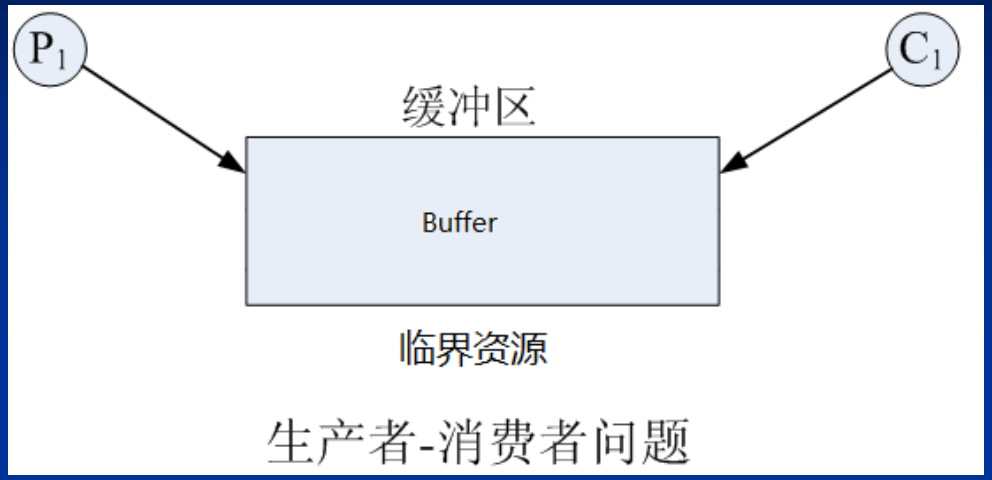
Consumer:
begin
      repeat
         wait( full ) ;

         nextc = Buffer(out);
         out := (out + 1 ) mod n ;

         signal( empty ) ;
         consume the item nextc ;
      until flase
end
Parend
```

(2) 问题思考

- 单生产者单消费者、单缓冲时



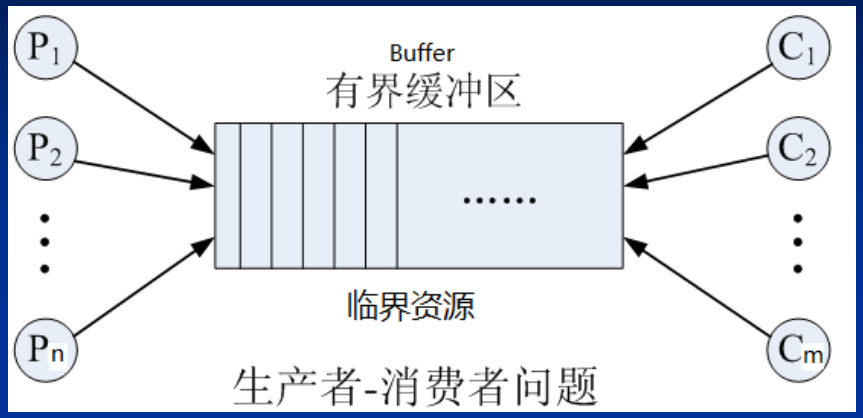
- 同例3

```
VAR empty, full: semaphore := 1, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
         repeat
            produce an item in nextp ;
            wait( empty ) ;
           Buffer(in) := nextp ;
            in := (in + 1 ) mod n ;
            signal( full ) ;
         until flase
   end

Consumer:
begin
      repeat
         wait( full ) ;
         nextc = Buffer(out);
         out := (out + 1 ) mod n ;
         signal( empty ) ;
         consume the item nextc ;
      until flase
end
Parend
```

(2) 问题思考

- 允许生产者写时，消费者可读

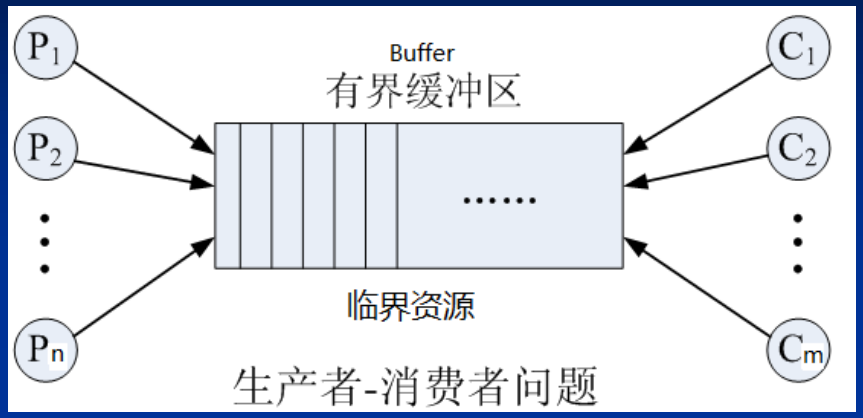


生产者–消费者标准程序

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
      in,out: integer := 0, 0 ;
      Buffer: array [0..n-1] of item ;
Parbegin
 Producer:
 begin
    repeat
       produce an item in nextp ;
       wait( empty ) ;
       wait( mutex ) ;
       Buffer(in) := nextp ;
        in := (in + 1 ) mod n ;
        signal( mutex ) ;
        signal( full ) ;
    until flase
 end
 Consumer:
 begin
     repeat
        wait( full ) ;
        wait( mutex ) ;
        nextc = Buffer(out);
        out := (out + 1 ) mod n ;
        signal( mutex ) ;
        signal( empty ) ;
        consume the item nextc ;
     until flase
 end
Parend
```

(2) 问题思考

- 允许生产者写时，消费者可读

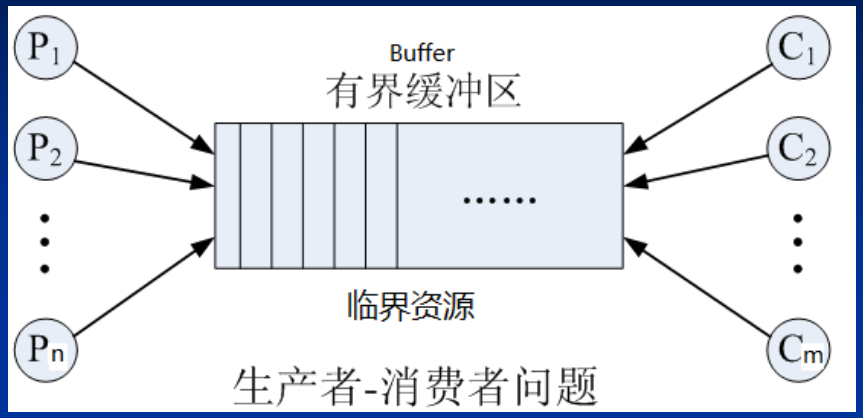



```
VAR mutexC, mutexP, empty, full: semaphore := 1,1, n, 0 ;
      in,out: integer := 0, 0 ;
      Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
        repeat
           produce an item in nextp ;
           wait( empty ) ;
           wait( mutexP ) ;
           Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;
           signal( mutexP ) ;
           signal( full ) ;
        until flase
   end

Consumer:
begin
     repeat
        wait( full ) ;
        wait( mutexC ) ;
        nextc = Buffer(out);
        out := (out + 1 ) mod n ;
        signal( mutexC ) ;
        signal( empty ) ;
        consume the item nextc ;
     until flase
end
Parend
```

(2) 问题思考

- 缓冲池无限大

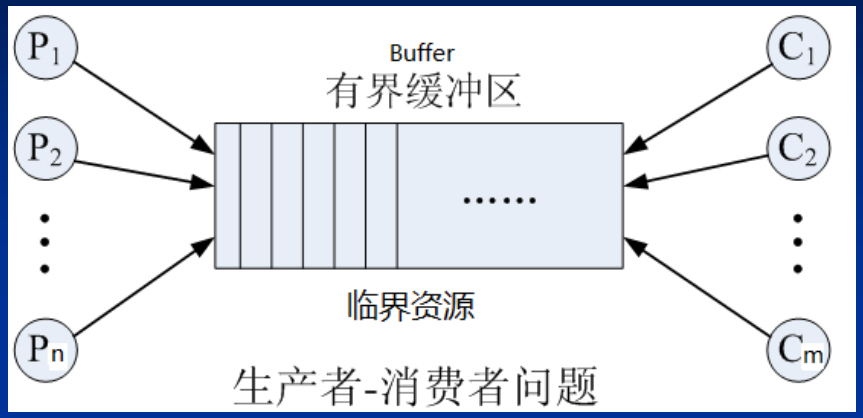


生产者-消费者标准程序

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
     repeat
          produce an item in nextp ;
          wait( empty ) ;
          wait( mutex ) ;
          Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;
           signal( mutex ) ;
           signal( full ) ;
     until flase
  end
  Consumer:
  begin
      repeat
          wait( full ) ;
          wait( mutex ) ;
          nextc = Buffer(out);
          out := (out + 1 ) mod n ;
          signal( mutex ) ;
          signal( empty ) ;
          consume the item nextc ;
      until flase
  end
Parend
```

(2) 问题思考

- 缓冲池无限大

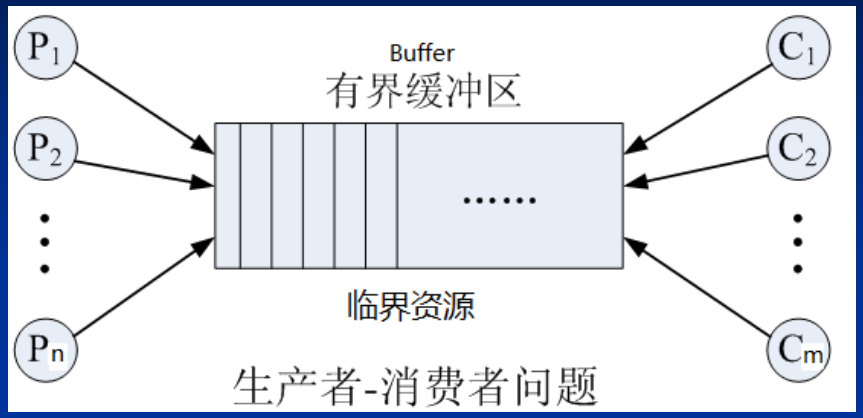



```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
     repeat
        produce an item in nextp ;
        wait( mutex ) ;
        Buffer(in) := nextp ;
        in := (in + 1 ) mod n ;
        signal( mutex ) ;
        signal( full ) ;
     until flase
   end
   Consumer:
   begin
     repeat
        wait( full ) ;
        wait( mutex ) ;
        nextc = Buffer(out);
        out := (out + 1 ) mod n ;
        signal( mutex ) ;

        consume the item nextc ;
     until flase
   end
Parend
```

(2) 问题思考

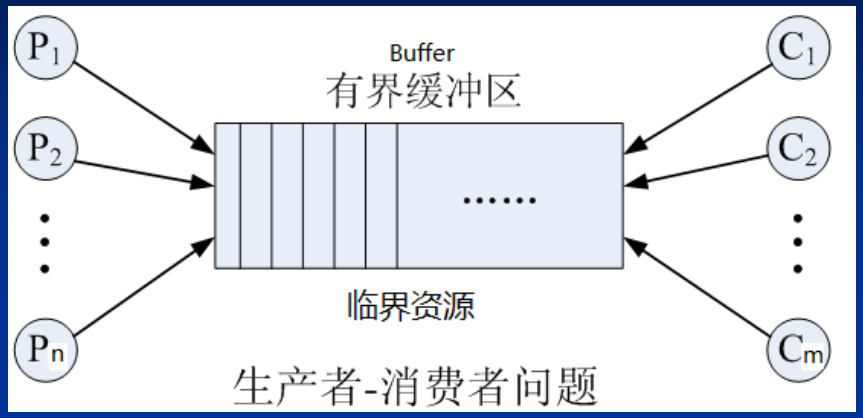
- 调整生产者wait顺序




```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
   Producer:
   begin
     repeat
          produce an item in nextp ;
          wait( mutex ) ;
          wait( empty ) ;
          Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;
           signal( mutex ) ;
           signal( full ) ;
       until flase
   end
   Consumer:
   begin
     repeat
          wait( full ) ;
          wait( mutex ) ;
          nextc = Buffer(out);
          out := (out + 1 ) mod n ;
          signal( mutex ) ;
          signal( empty ) ;
          consume the item nextc ;
       until flase
   end
Parend
```

(2) 问题思考

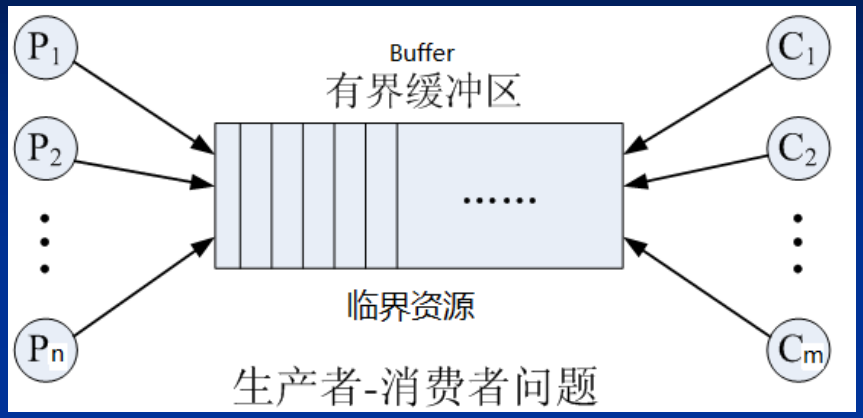
- 调整消费者wait顺序




```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
 Producer:
 begin
   repeat
     produce an item in nextp ;
      wait( empty ) ;
      wait( mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1 ) mod n ;
      signal( mutex ) ;
      signal( full ) ;
    until flase
 end
 Consumer:
 begin
    repeat
       wait( mutex ) ;
       wait( full ) ;
       nextc = Buffer(out);
       out := (out + 1 ) mod n ;
       signal( mutex ) ;
       signal( empty ) ;
       consume the item nextc ;
     until flase
 end
Parend
```

(2) 问题思考

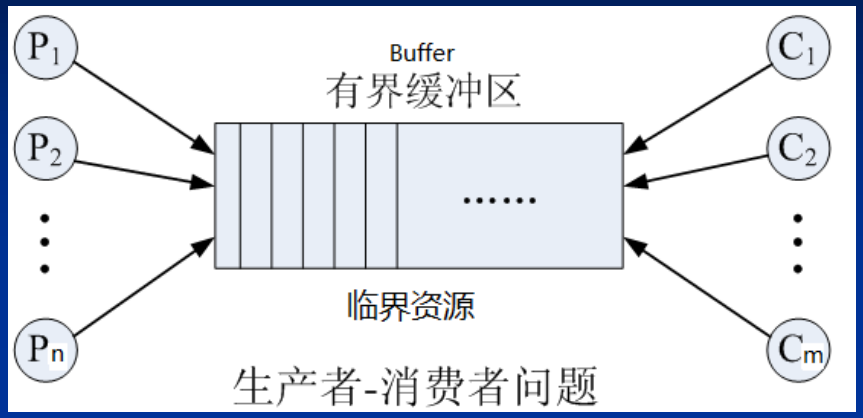
- 调整wait顺序




```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
    in,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
 Producer:
 begin
   repeat
     produce an item in nextp ;
     wait( mutex ) ;
     wait( empty ) ;
     Buffer(in) := nextp ;
     in := (in + 1 ) mod n ;
     signal( mutex ) ;
     signal( full ) ;
   until flase
 end
 Consumer:
 begin
   repeat
     wait( mutex ) ;
     wait( full ) ;
     nextc = Buffer(out);
     out := (out + 1 ) mod n ;
     signal( mutex ) ;
     signal( empty ) ;
     consume the item nextc ;
   until flase
 end
Parend
```

(2) 问题思考

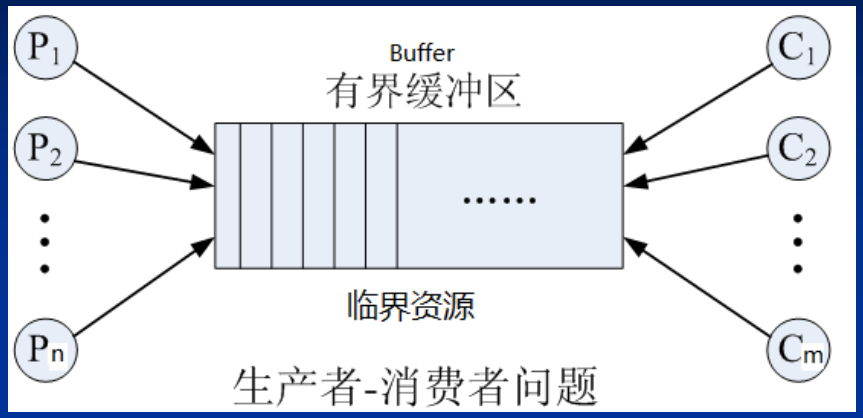
- 调整signal顺序




```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
      repeat
          produce an item in nextp ;
          wait( empty ) ;
          wait( mutex ) ;
          Buffer(in) := nextp ;
          in := (in + 1 ) mod n ;
          signal( mutex ) ;
          signal( full ) ;
      until flase
    end
    Consumer:
    begin
      repeat
          wait( full ) ;
          wait( mutex ) ;
          nextc = Buffer(out);
          out := (out + 1 ) mod n ;
          signal( empty ) ;
          signal( mutex ) ;
          consume the item nextc ;
      until flase
     end
Parend
```

(2) 问题思考

- 每个消费者消费每个消息1次



- 如何实现？

留给大家课后解决

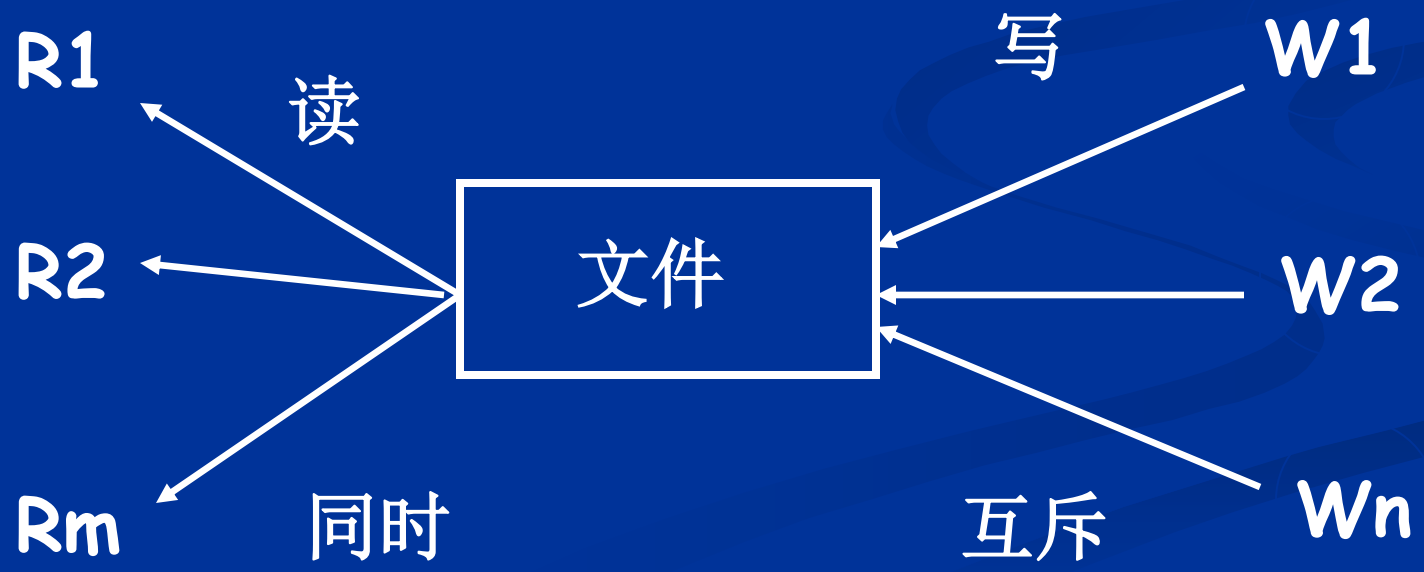
生产者-消费者标准程序

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
     repeat
          produce an item in nextp ;
          wait( empty ) ;
          wait( mutex ) ;
          Buffer(in) := nextp ;
           in := (in + 1 ) mod n ;
           signal( mutex ) ;
           signal( full ) ;
     until flase
  end
  Consumer:
  begin
      repeat
          wait( full ) ;
          wait( mutex ) ;
          nextc = Buffer(out);
          out := (out + 1 ) mod n ;
          signal( mutex ) ;
          signal( empty ) ;
          consume the item nextc ;
      until flase
  end
Parend
```

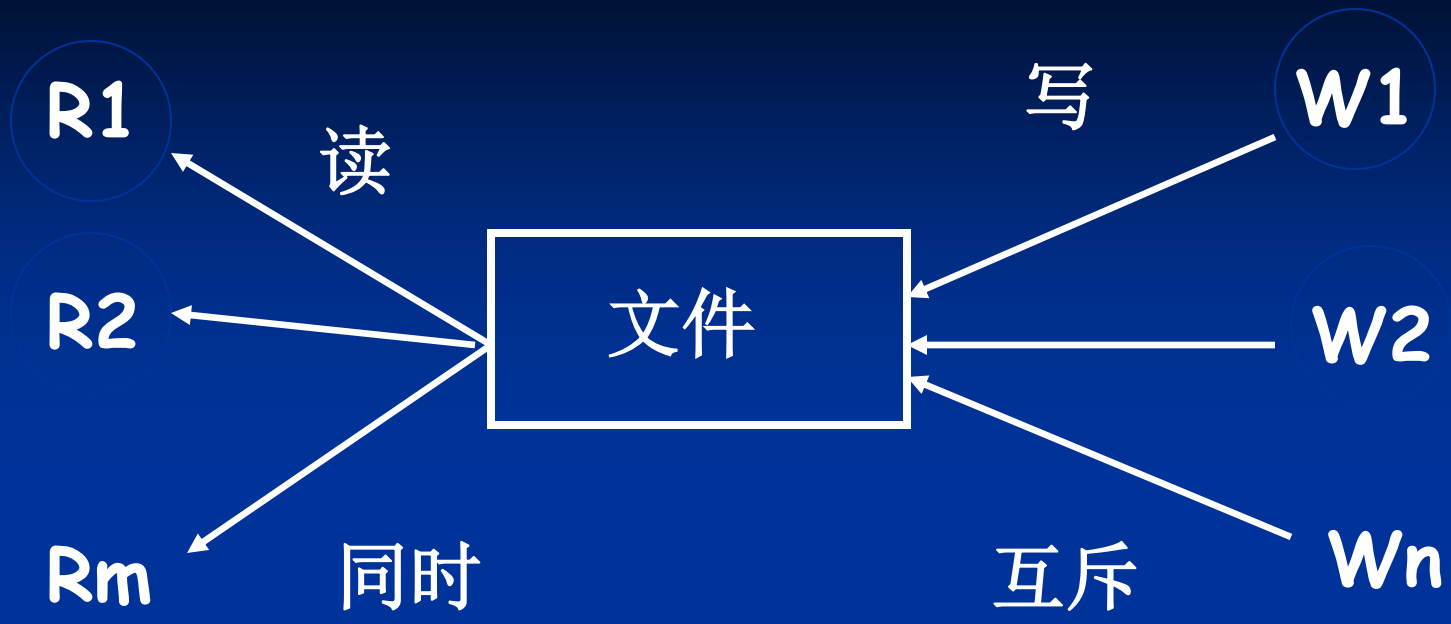
2.4 经典进程同步问题
2.4.2 读者-写者问题

(1) 读者写者问题

多个读者、多个写者共享资源（例如文件、数据等），允许多个读者同时访问资源，写者写时不允许其他进程读或写（互斥访问资源）。请用记录型信号量进行同步。



进程资源共享关系和同步关系分析



- 读者进程：不控制
- 写者进程：互斥
- 读者写者顺序：无

读者-写者并发程序实现

```
VAR wmutex: semaphore := 1 ;
Parbegin
 Reader:
 begin
    repeat
        wait( wmutex ) ;
        perform read operation ;
        signal( wmutex ) ;
     until flase
 end
 Writer:
 begin
     repeat
        wait( wmutex ) ;
        perform write operation;
        signal( wmutex ) ;
     until flase
 end
Parend
```

什么问题？

读者-写者并发程序实现

```
VAR wmutex: semaphore := 1 ;
        readcount: integer := 0 ;
Parbegin
  Reader:
  begin
      repeat
          if  ( readcount = 0 ) wait( wmutex ) ;
          readcount := readcount + 1 ;
          perform read operation ;
          readcount := readcount - 1 ;
          if ( readcount = 0 ) signal( wmutex ) ;
       until flase
  end
  Writer:
  begin
      repeat
          wait( wmutex ) ;
          perform write operation;
          signal( wmutex ) ;
      until flase
  end
Parend
```

- 什么问题？

读者-写者并发程序实现

```
VAR wmutex: semaphore := 1 ;
        readcount: integer := 0 ;
Parbegin
  Reader:
  begin
     repeat
          if ( readcount = 0 ) wait( wmutex ) ;
          readcount := readcount + 1 ;
          perform read operation ;
          readcount := readcount - 1 ;
          if ( readcount = 0 ) signal( wmutex ) ;
       until flase
  end
  Writer:
  begin
      repeat
          wait( wmutex ) ;
          perform write operation;
          signal( wmutex ) ;
      until flase
  end
Parend
```

读者-写者标准程序

```
VAR rmutex,wmutex: semaphore := 1, 1 ;
        readcount: integer := 0 ;
Parbegin
  Reader:
  begin
     repeat
          wait( rmutex ) ;
          if ( readcount = 0 ) wait( wmutex ) ;
          readcount := readcount + 1 ;
          signal( rmutex ) ;
          perform read operation ;
          wait( rmutex ) ;
          readcount := readcount - 1 ;
          if ( readcount = 0 ) signal( wmutex ) ;
          signal( rmutex ) ;
       until flase
  end
  Writer:
  begin
      repeat
          wait( wmutex ) ;
          perform write operation;
          signal( wmutex ) ;
      until flase
  end
Parend
```

读者-写者标准程序

```
VAR rmutex,wmutex: semaphore := 1, 1 ;
        readcount: integer := 0 ;
Parbegin
 Reader:
 begin
    repeat
        wait( rmutex ) ;
        if ( readcount = 0 ) wait( wmutex ) ;
        readcount := readcount + 1 ;
        signal( rmutex ) ;
        perform read operation ;
        wait( rmutex ) ;
        readcount := readcount - 1 ;
        if ( readcount = 0 ) signal( wmutex ) ;
        signal( rmutex ) ;
      until flase
 end
 Writer:
 begin
     repeat
        wait( wmutex ) ;
        perform write operation;
        signal( wmutex ) ;
     until flase
 end
Parend
```

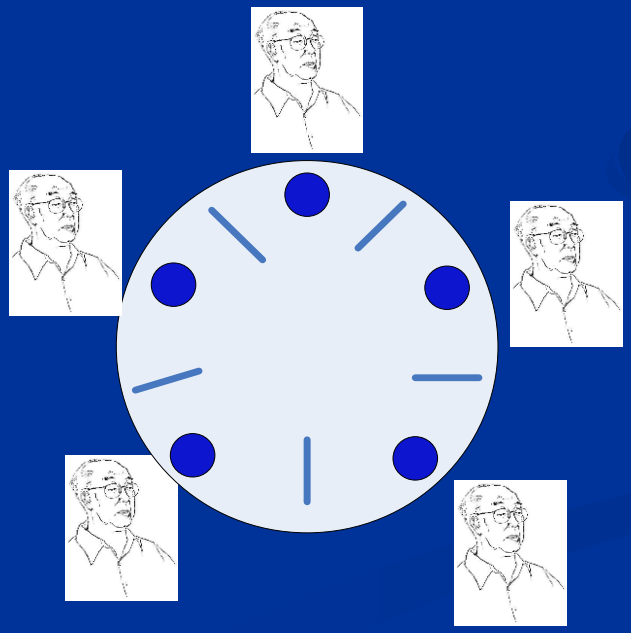
(2) 标准程序的2个问题

- 执行wait和signal的进程
- 临界区设置

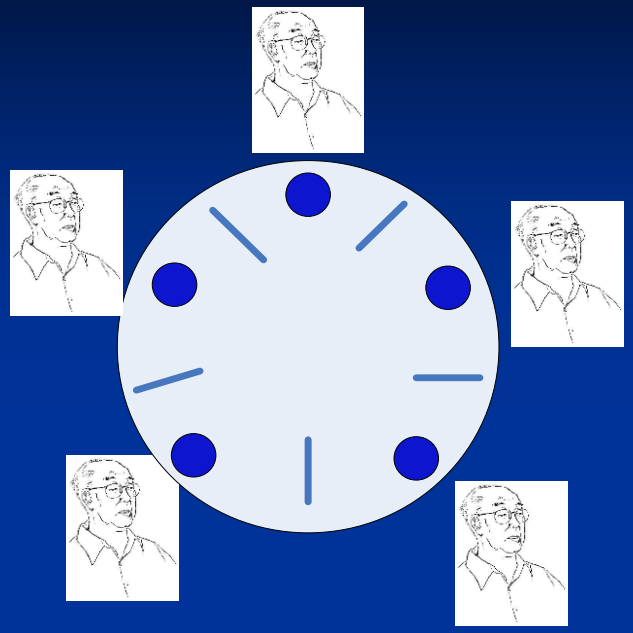
2.4 经典进程同步问题
2.4.3 哲学家进餐问题

(1) 哲学家进餐问题

5位哲学家围绕圆桌而坐，反复思考和进餐。但是只有5只碗和筷子，放置如图所示，只有当哲学家同时拿起碗边的2只筷子时，才能进餐。请用记录型信号量进行同步。

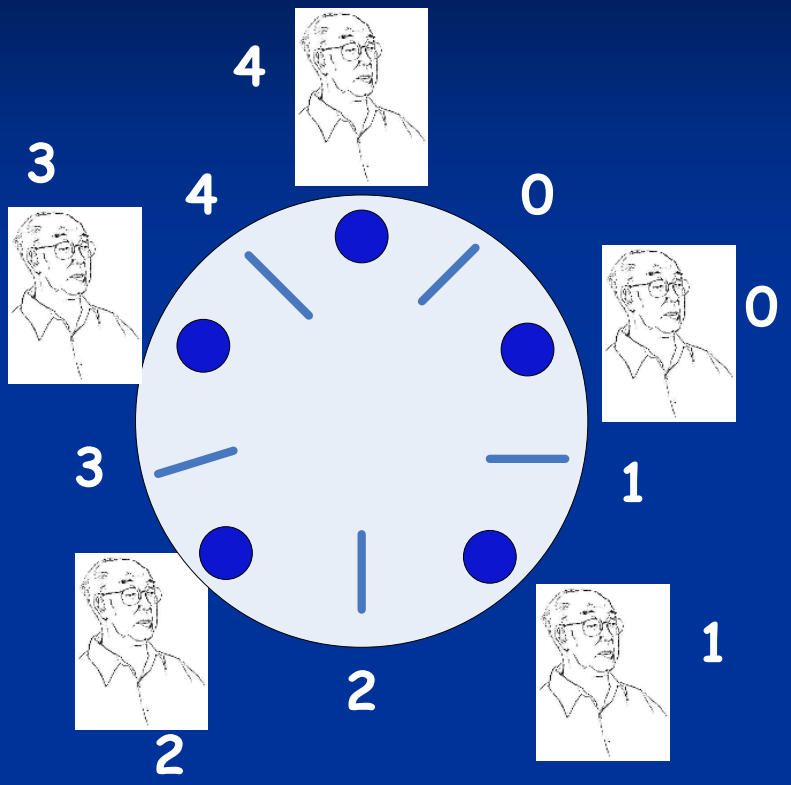


进程资源共享关系和同步关系分析



- 哲学家进程：互斥（筷子）

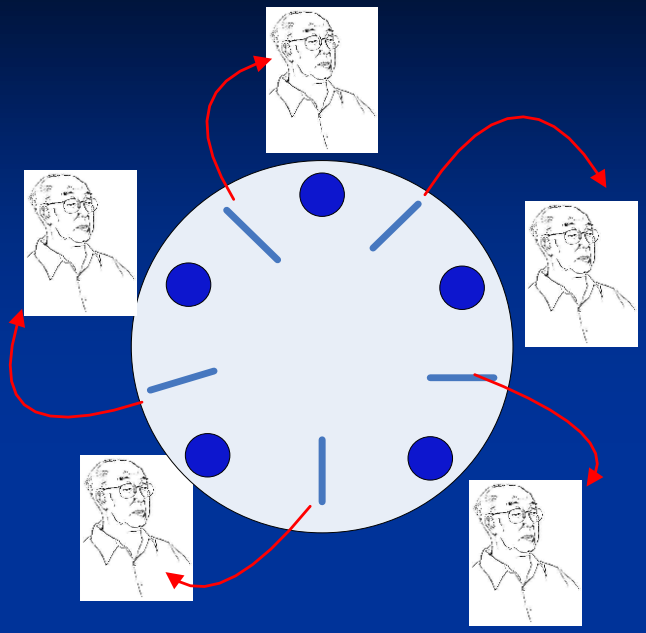
哲学家进餐问题实现

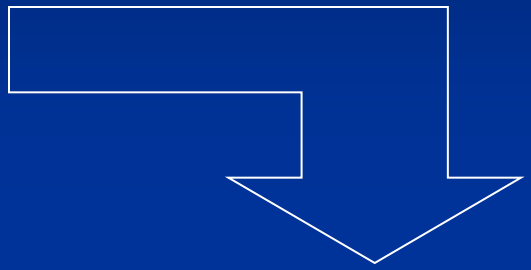


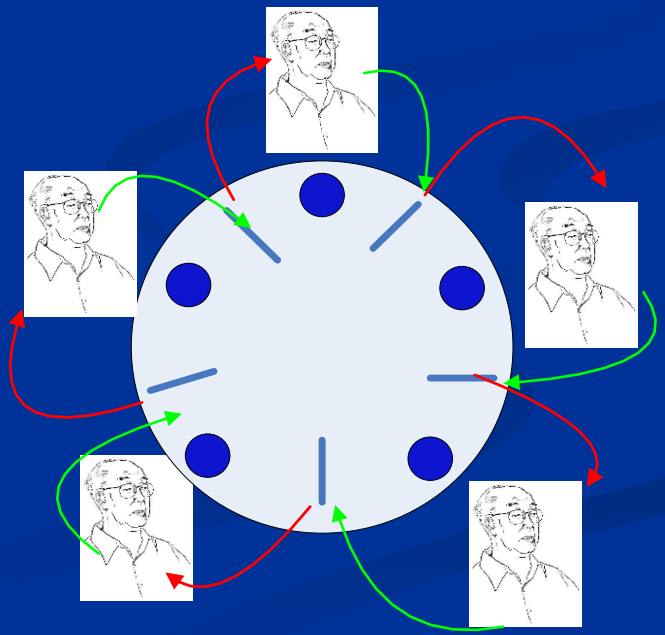

```
VAR chopsticks: array [0..4] of semaphore := 1,1,1,1,1 ;
Parbegin
 philosopheri:
 begin
    repeat
        wait( chopsticks[ i ] ) ;
        wait( chopsticks[ (i+1) mod 5 ] );
        eat ;
        signal( chopsticks[ i ] ) ;
        signal( chopsticks[ (i+1) mod 5 ] );
     think
     until flase
 end
Parend
```

- 问题：正确吗？

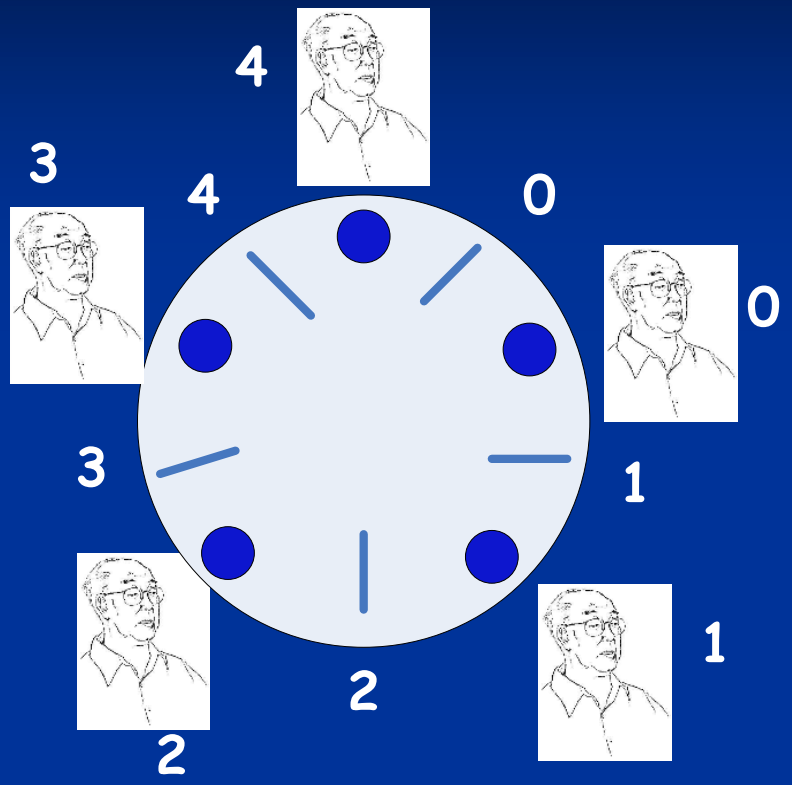
(2) 哲学家进餐程序的问题







哲学家进餐标准程序




```
VAR chopsticks: array [0..4] of semaphore := 1,1,1,1,1 ;
Parbegin
    philosopher0-3:
    begin
       repeat
            wait( chopsticks[ i ] ) ;
            wait( chopsticks[ (i+1) mod 5 ] );
            eat ;
           signal( chopsticks[ i ] ) ;
           signal( chopsticks[ (i+1) mod 5 ] );
         think
          until flase
    end
 philosopher4:
 begin
     repeat
        wait( chopsticks[ (i+1) mod 5 ] );
        wait( chopsticks[ i ] ) ;
        eat ;
        signal( chopsticks[ i ] ) ;
        signal( chopsticks[ (i+1) mod 5 ] );
        think
    until flase
 end
Parend
```

- 原则：按序申请资源

2.4 经典进程同步问题
2.4.4 理发师问题(课后思考题)

(1) 睡眠理发师问题

理发店有一位理发师，一把理发椅和n把用来等候理发的椅子。如果没有顾客，则更好师便在理发椅上睡觉，顾客到来时，如理发师闲则理发，否则如有空椅则坐等，没有空椅则离开，编写程序实现理发师和顾客程序，实现进程控制

2.4 经典进程同步问题
2.4.5 信号量集机制

(1) AND型信号量集机制

- 基本思想：
  - 将进程在整个运行过程中需要的所有资源，一次性全部地分配给进程，待进程使用完后再一起释放。
  - 对若干个临界资源的分配，采取原子操作方式要么全部分配到进程，要么一个也不分配。
  - 为此，在wait操作中，增加了一个“AND”条件，故称为AND同步。

2.4.5 信号量集机制

(1) AND型信号量集机制

- 实现：

```
type semaphore = record
    value: integer ;                    // 资源数量
    L: list of process ;                // 阻塞进程队列
end

function Swait ( var S1,S2,..,Sn ： Semaphore )
Begin
   if ( S1 >= 1 and ... and Sn >= 1 )
      for i := 1 to n do
         Si := Si -1 ;
   else
      Place the process in the queue associated with the first Si found with Si < 1, and
      goto the beginning of Swait.
end
function Ssignal(var S1, S2, .., Sn ： Semaphore )
Begin
   for i:=1 to n do
     Si := Si +1
     Remove all process waiting in the queue associated with Si into the ready queue.
end
```

2.4.5 信号量集机制

(1) AND型信号量集机制

- 应用示例1：生产者消费者问题

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
        in,out: integer := 0, 0 ;
        Buffer: array [0..n-1] of item ;
Parbegin
 Producer:
 begin
   repeat
     produce an item in nextp ;
     Swait( empty, mutex ) ;
     Buffer(in) := nextp ;
     in := (in + 1 ) mod n ;
     Ssignal( mutex, full ) ;
    until flase
 end
 Consumer:
 begin
     repeat
        Swait( full,mutex ) ;
        nextc = Buffer(out);
        out := (out + 1 ) mod n ;
        Ssignal( mutex,epty ) ;
        consume the item nextc ;
     until flase
 end
Parend
```

2.4.5 信号量集机制

(1) AND型信号量集机制

- 应用示例2：哲学家进餐问题

```
VAR chopsticks: array [0..4] of semaphore := 1,1,1,1,1 ;
Parbegin
 philosopheri:
 begin
    repeat
        Swait( chopsticks[ i ], chopsticks[ (i+1) mod 5 ] ) ;
        eat ;
        Ssignal( chopsticks[ i ], chopsticks[ (i+1) mod 5 ] ) ;
        think
     until flase
 end
Parend
```

2.4 经典进程同步问题
2.4.5 信号量集机制

(2) 一般信号量集机制

- 基本思想：
  - 在AND型信号量基础上，一次可申请多个单位资源。

2.4.5 信号量集机制

(2) 一般信号量集机制

- 实现：

```
type semaphore = record
    value: integer ;                    // 资源数量
    L: list of process ;                // 阻塞进程队列
end

function Swait ( var S1,t1,d1,S2,t2,d2..,Sn,tn,dn ： Semaphore )
Begin
   if ( S1 >= t1 and … and Sn >= tn )
      for i := 1 to n do
         Si := Si -dn ;
   else
      Place the process in the queue associated with the first Si found with Si < ti, and
      goto the beginning of Swait.
end
function Ssignal(var S1, d1,S2,d2, .., Sn,dn ： Semaphore )
Begin
   for i:=1 to n do
     Si := Si +dn
     Remove all process waiting in the queue associated with Si into the ready queue.
end
```

2.4.5 信号量集机制

(2) 一般信号量集机制

- 应用实例：读者写者问题

RN: 允许读者进程上限

```
VAR RN: integer ;
        L,mutex: semaphore := RN, 1 ;
Parbegin
 Reader:
 begin
    repeat
        Swait( L,1,1,  mutext,1,0) ;
        perform read operation ;
        Ssignal( L, 1 ) ;
     until flase
 end
 Writer:
 begin
     repeat
        Swait( mutex,1,1, L,RN,0 ) ;
        perform write operation;
        Ssignal( mutex,1 ) ;
     until flase
 end
Parend
```

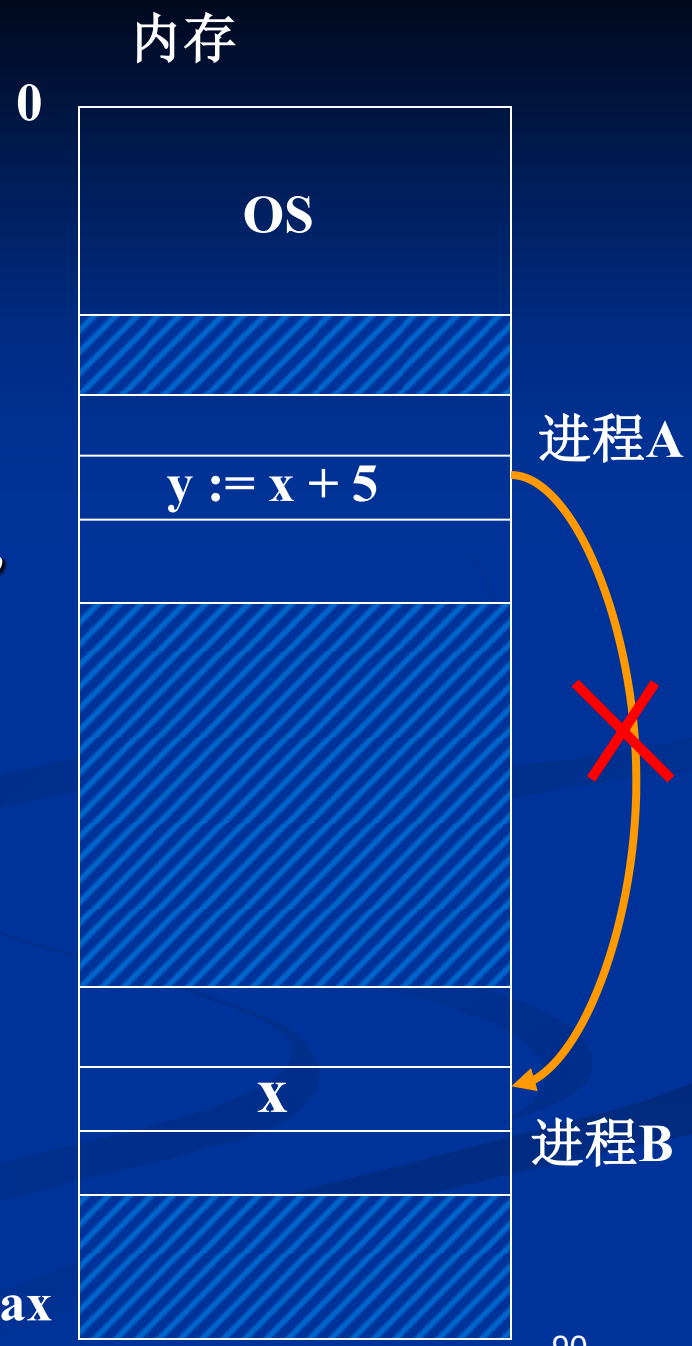
- 问题分析
  可否再简化程序？

2.5 进程通信

- 进程间通信的基本概念

  Inter Process Communication, IPC

  进程通信指进程之间的信息交换，其所交换的信息量，少者是一个状态或数值，多者则是成千上万个字节。



2.5 进程通信

- 高级通信
  - 通信链路：
    - 点对点/多点/广播
    - 单向/双向
    - 有容量（链路带缓冲区）/无容量（发送方和接收方需自备缓冲区）
  - 数据格式：
    - 字节流：各次发送之间的分界，在接收时不被保留，没有格式
    - 报文：各次发送之间的分界，在接收时被保留，通常有格式（如表示类型），定长/不定长报文，可靠报文/不可靠报文
  - 收发操作的同步方式
    - 发送阻塞（直到被链路容量或接收方所接受）和不阻塞（失败时立即返回）
    - 接收阻塞（直到有数据可读）和不阻塞（无数据时立即返回）
    - 由事件驱动收发：在允许发送或有数据可读时，才做发送和接收操作

2.5 进程通信

2.5.1 共享存储器系统

- 基本思想

  相互通信的进程共享数据结构或共享存储区，进程之间通过这些空间进行通信。

- 实现过程

  ① 进程申请一片内存，拿到内存描述符；

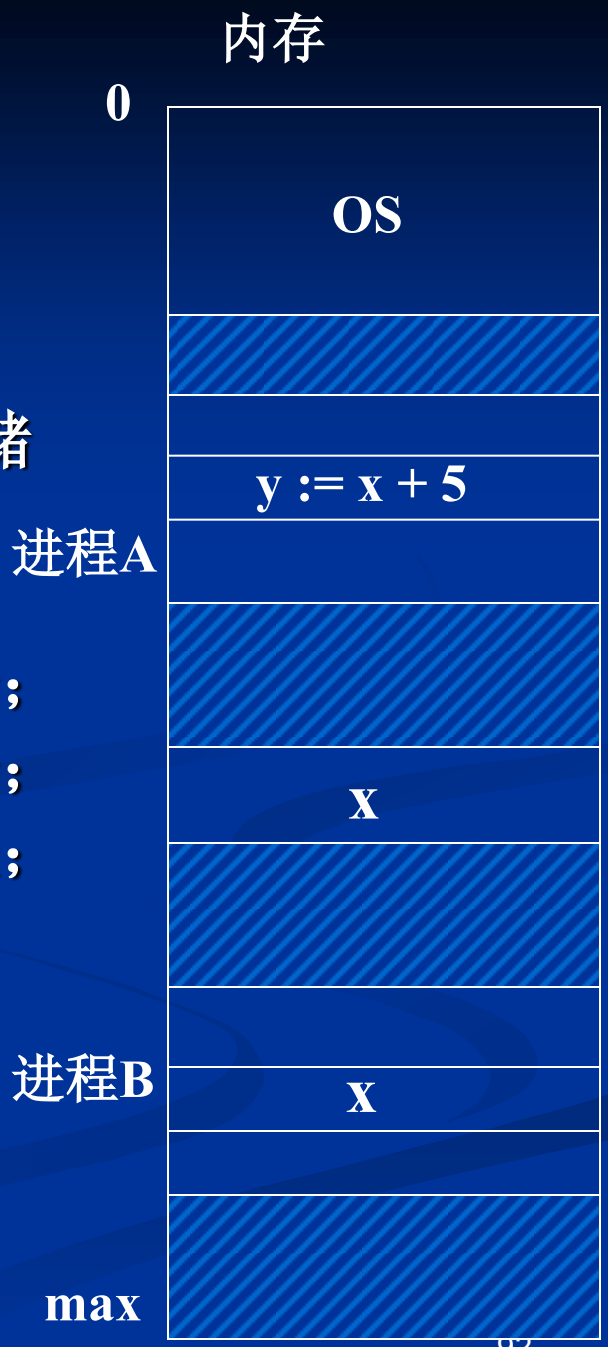
  ② 如果已分配过，直接获得内存描述符；

  ③ 利用内存描述符将内存连接到本进程；

  ④ 读写

- 共享存储区的访问同步

  共享存储区的资源共享问题



2.5 进程通信
2.5.1 共享存储器系统

- 分类

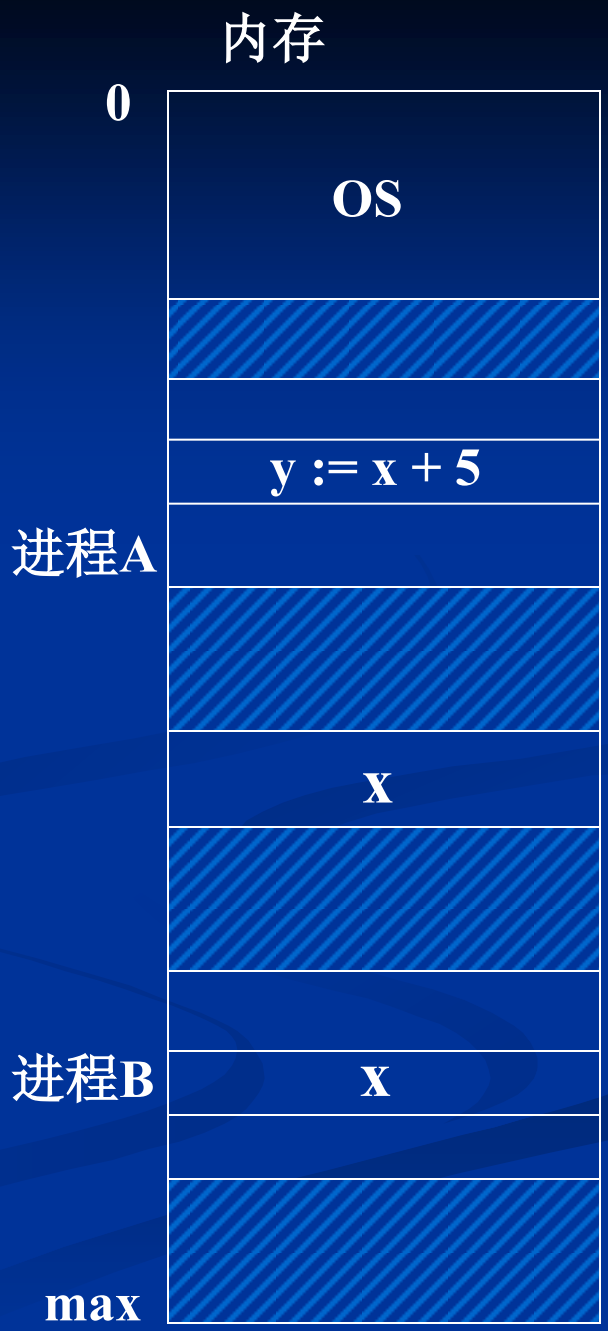
  基于共享数据结构的通信方式

  基于共享存储区的通信方式

- 特点

  适合大量的数据快速交换；

  程序员负担重



2.5 进程通信
2.5.1 消息通信系统

- 基本思想

  进程间的数据交换，是以格式化的消息(message)为单位进行通信。
- 直接通信：信息直接传递给接收方
  - 在发送时，指定接收方的地址或标识，也可以指定多个接收方或广播式地址
  - 在接收时，允许接收来自任意发送方的消息，并在读出消息的同时获取发送方的地址
- 间接通信：借助于收发双方进程之外的共享数据结构作为通信中转，如消息队列。

  通常收方和发方的数目可以是任意的

2.5 进程通信

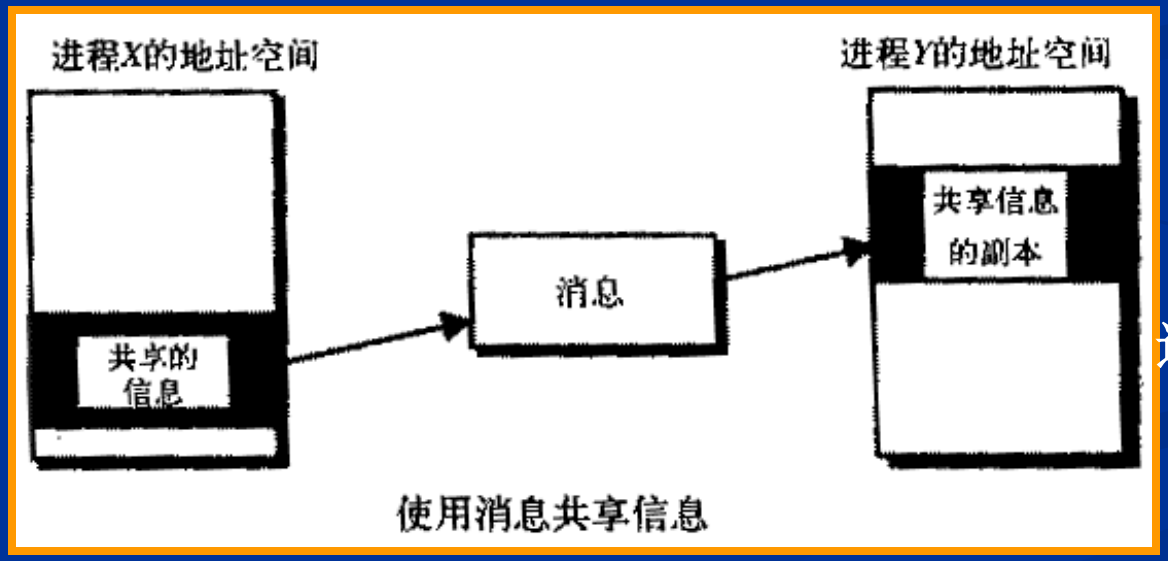
2.5.1 消息通信系统

(1) 直接通信方式

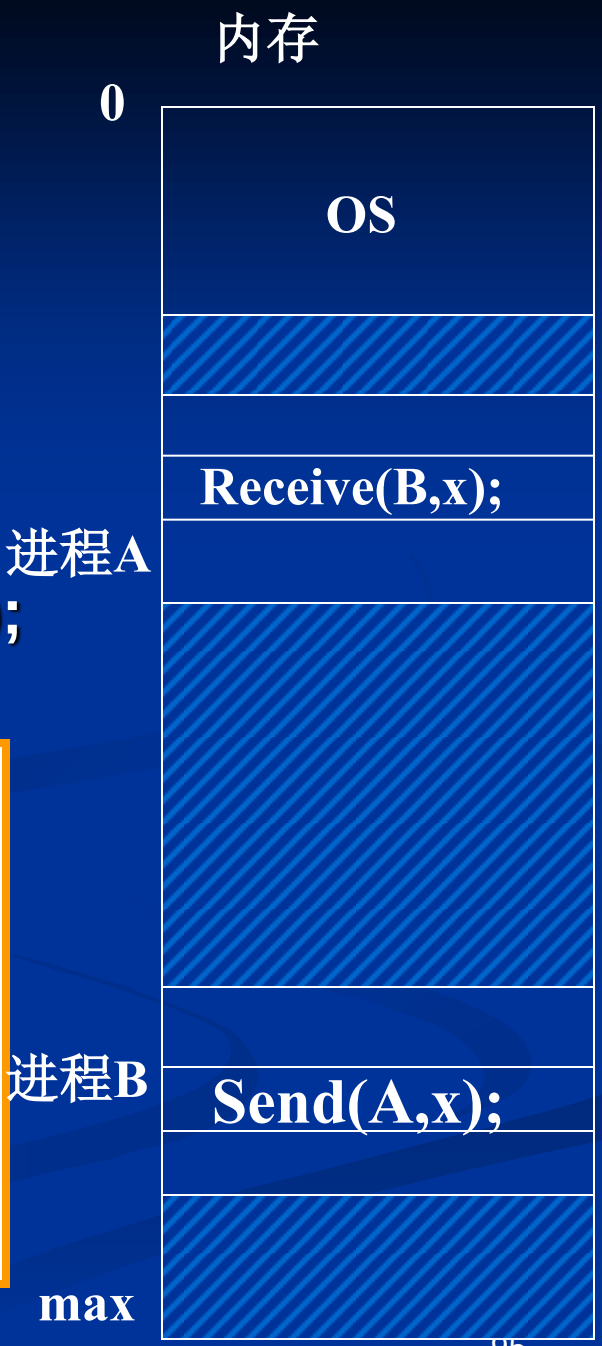
OS提供两个通信原语：

① 发送进程： **Send(receiver,msg);**

② 接收进程： **Receive (sender,msg);**



使用消息共享信息



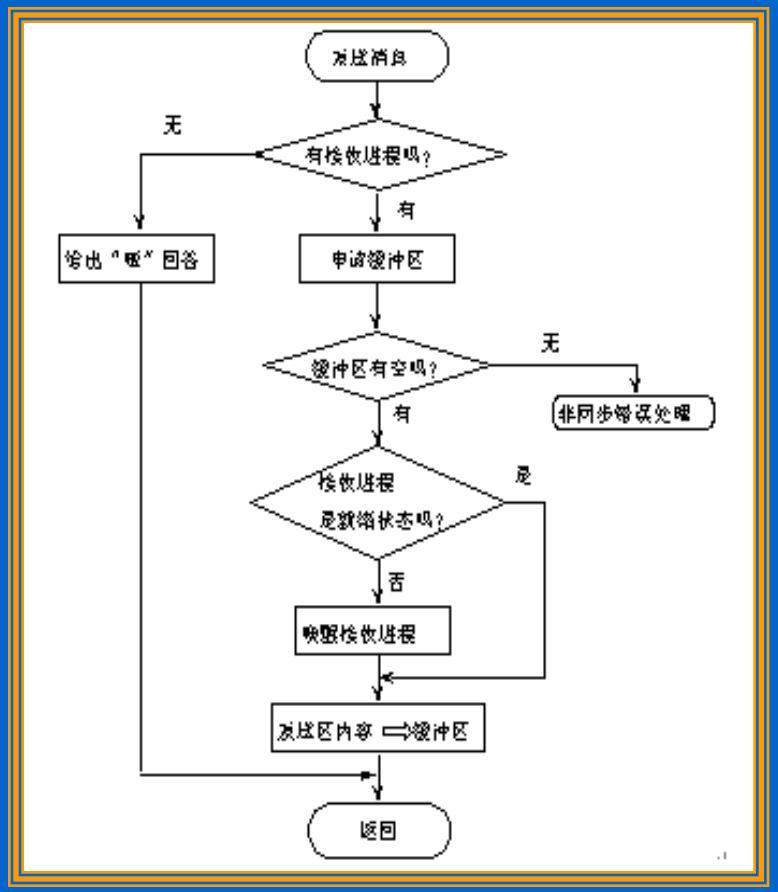
2.5 进程通信
2.5.2 消息通信系统

- 消息的基本结构

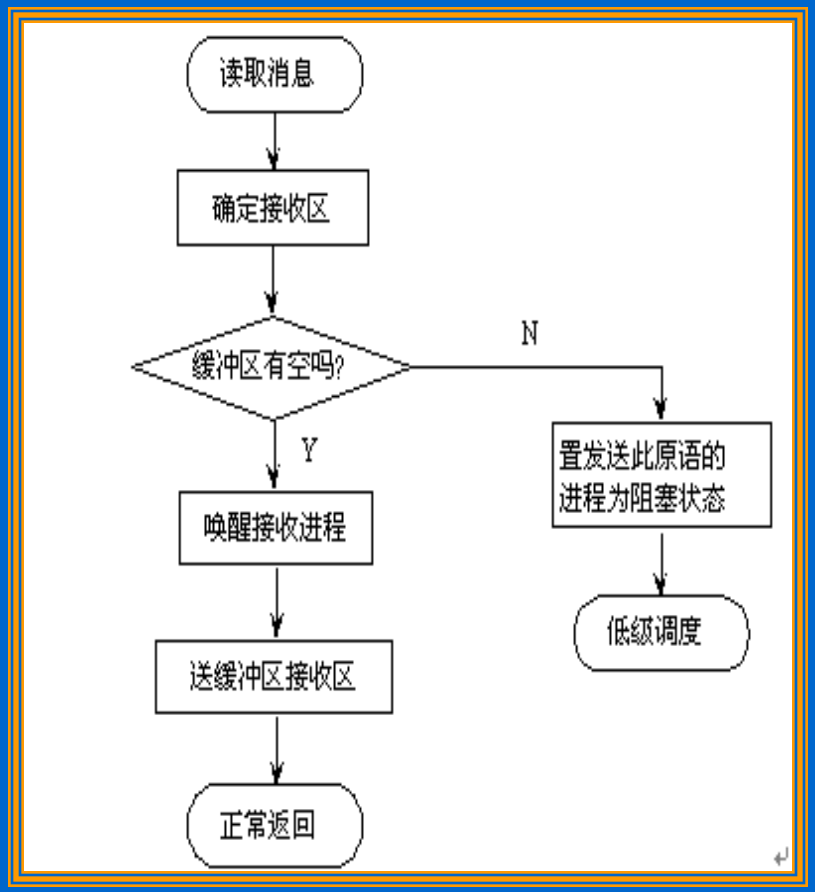
```
type message buffer=record
                        sender ; 发送者进程标识符
                        size   ; 消息长度
                        text   ; 消息正文
                        next   ; 指向下一个消息缓冲区的指针
                    end
```

2.5 进程通信
2.5.2 消息通信系统

- 发送消息的流程



- 接收消息的流程



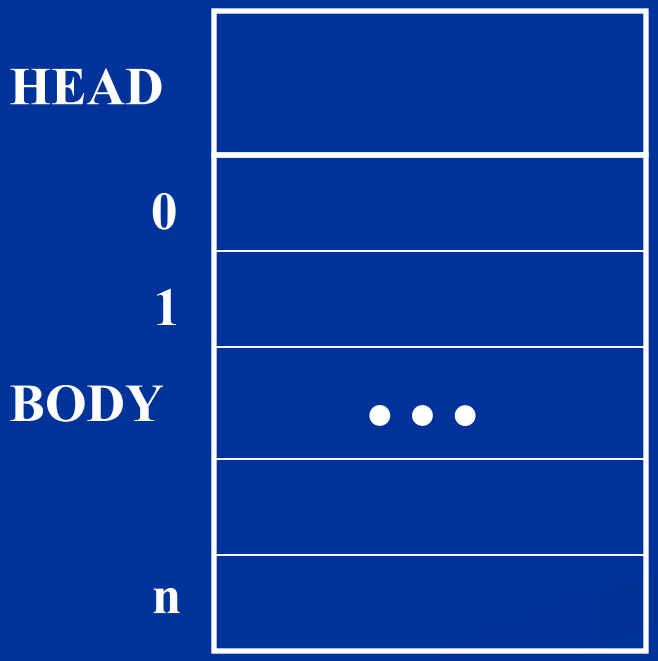
2.5 进程通信
2.5.2 消息通信系统

(2) 间接通信方式:信箱通信

① 发送进程：Send(mailbox,msg);

② 接收进程：Receive(mailbox,msg);

- 信箱（mailbox)





2.5 进程通信
2.5.2 消息通信系统

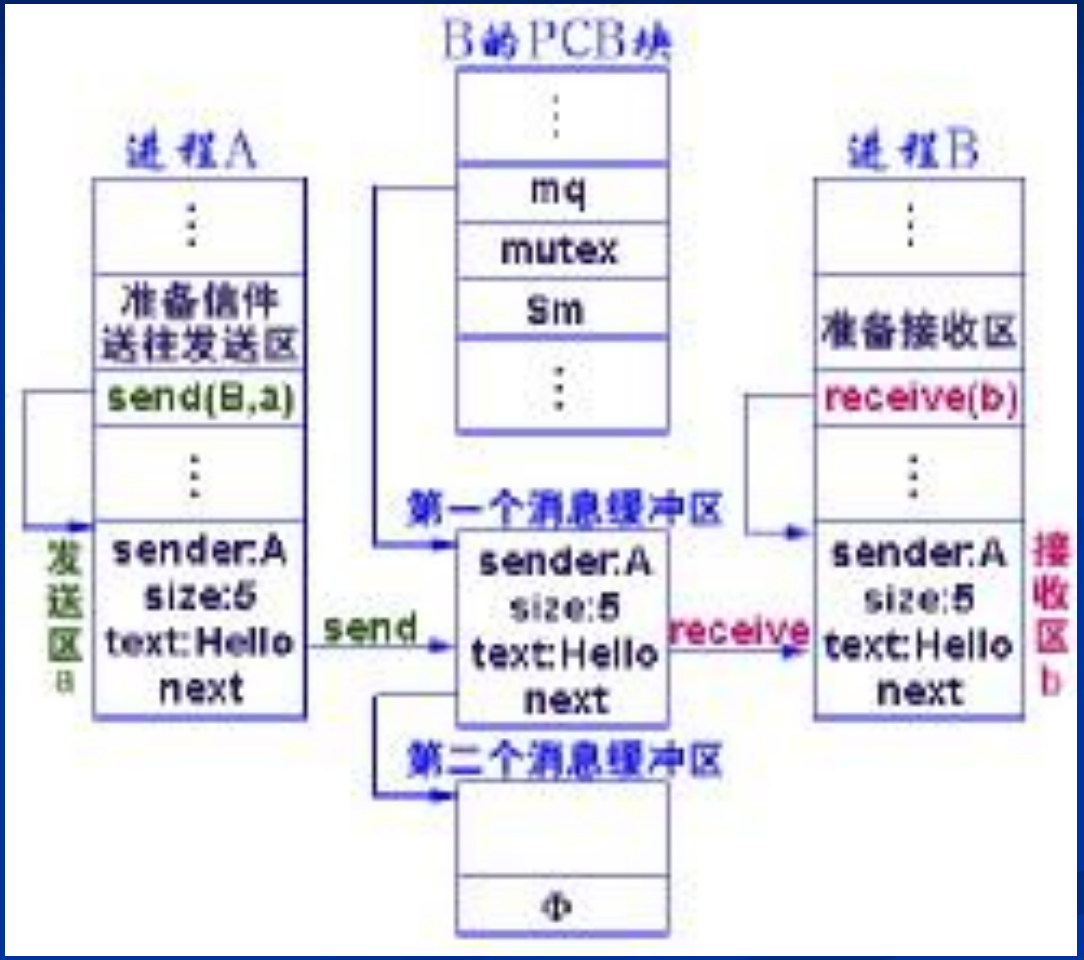
- 信箱分类
  - 公有信箱；
  - 私有信箱；
  - 共享信箱；
- 特点
  - 操作系统隐藏通信细节；
  - 通信程序简单；
  - 可以跨网络传输；
  - 灵活
- 进程同步
  - 发送阻塞、接收阻塞；
  - 发送不阻塞、接收阻塞；
  - 发送不阻塞、接收不阻塞；



2.5 进程通信
2.5.2 消息通信系统

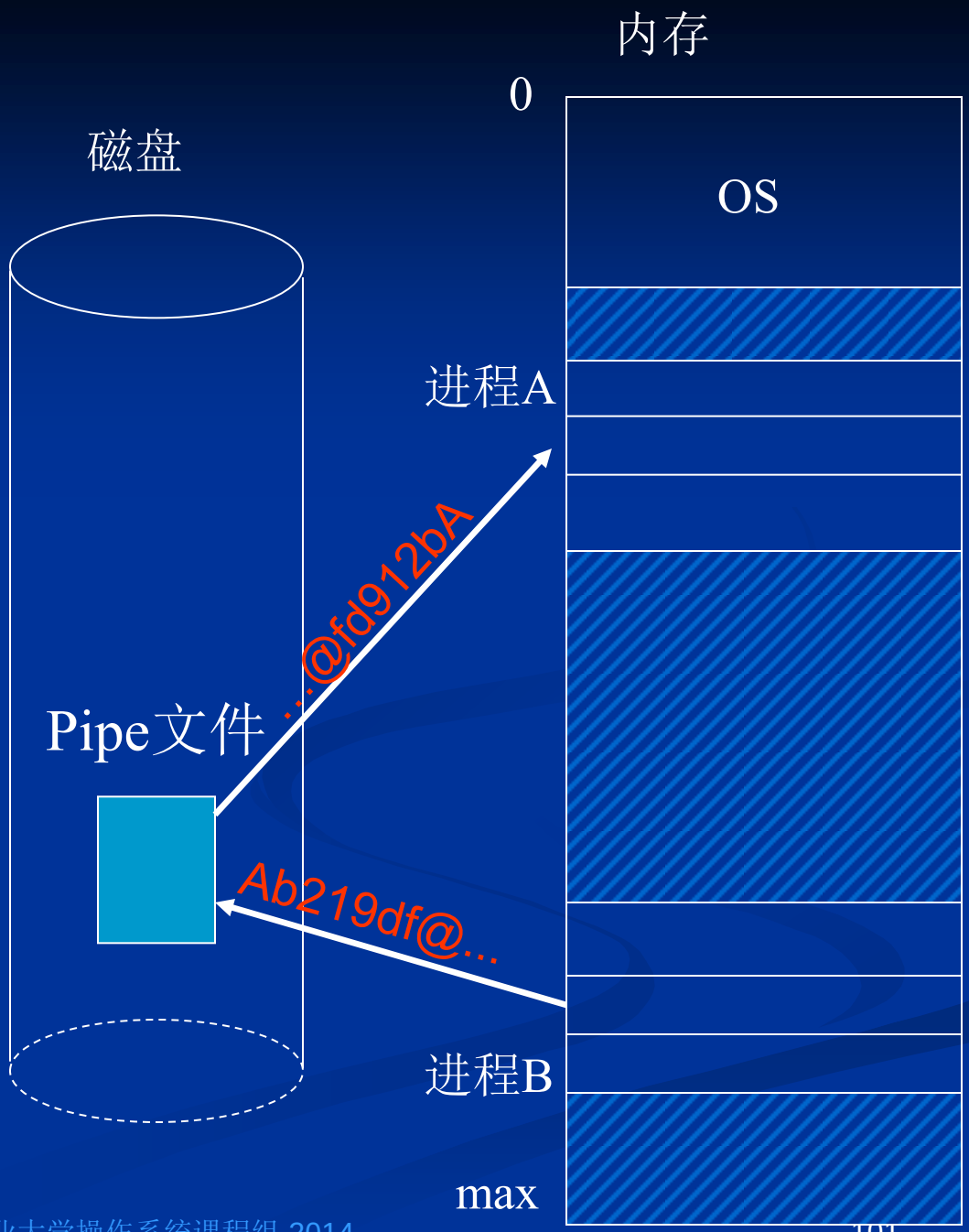
(3) 消息缓冲队列通信

Hansan, RC4000



2.5 进程通信
2.5.3 管道通信系统

- 基本思想
  在磁盘创造一个文件；
  大小固定，如4kB；
  写连续字符流；
  读连续字符流；
- UNIX首创
- 实现技术
  对管道文件的资源共享：
  同步、互斥



2.5 进程通信
2.5.3 管道通信系统

- **Windows操作系统的管道通信**
  - Widows server 2003提供无名管道和命名管道两种管道机制。
  - 利用CreatePipe可创建无名管道并得到两个读写句柄；
  - 利用ReadFile和WriteFile可并行无名管道的读写。
- **CreatePipe的调用格式：**

```
BOOL CreatePipe( PHANDLE hReadPipe, // 读句柄;
PHANDLE hWritePipe, // 写句柄;
LPSECURITY_ATTRIBUTES lpPipeAttributes, // 安全属性指针;
DWORD nSize         // 管道缓冲区字节数;
);
```

2.6 线程
2.6.1 线程的基本概念

(1) 线程的引入

- 传统进程的两个基本属性
  - 拥有资源的独立单位

    每个进程分配一虚拟地址空间，保存进程映像，控制一些资源（文件，I/O设备），有状态、优先级
  - 调度和分派的基本单位

    分配CPU
- 导致的不利后果
  - 身形笨重，管理和运行资源开销大。
  - 限制并发度，多道程序设计道数不能太高。



2.6 线程

(1) 线程的引入

- 传统进程的两个基本属性分解
  - 拥有资源的独立单位
  - 调度和分派的基本单位

进程

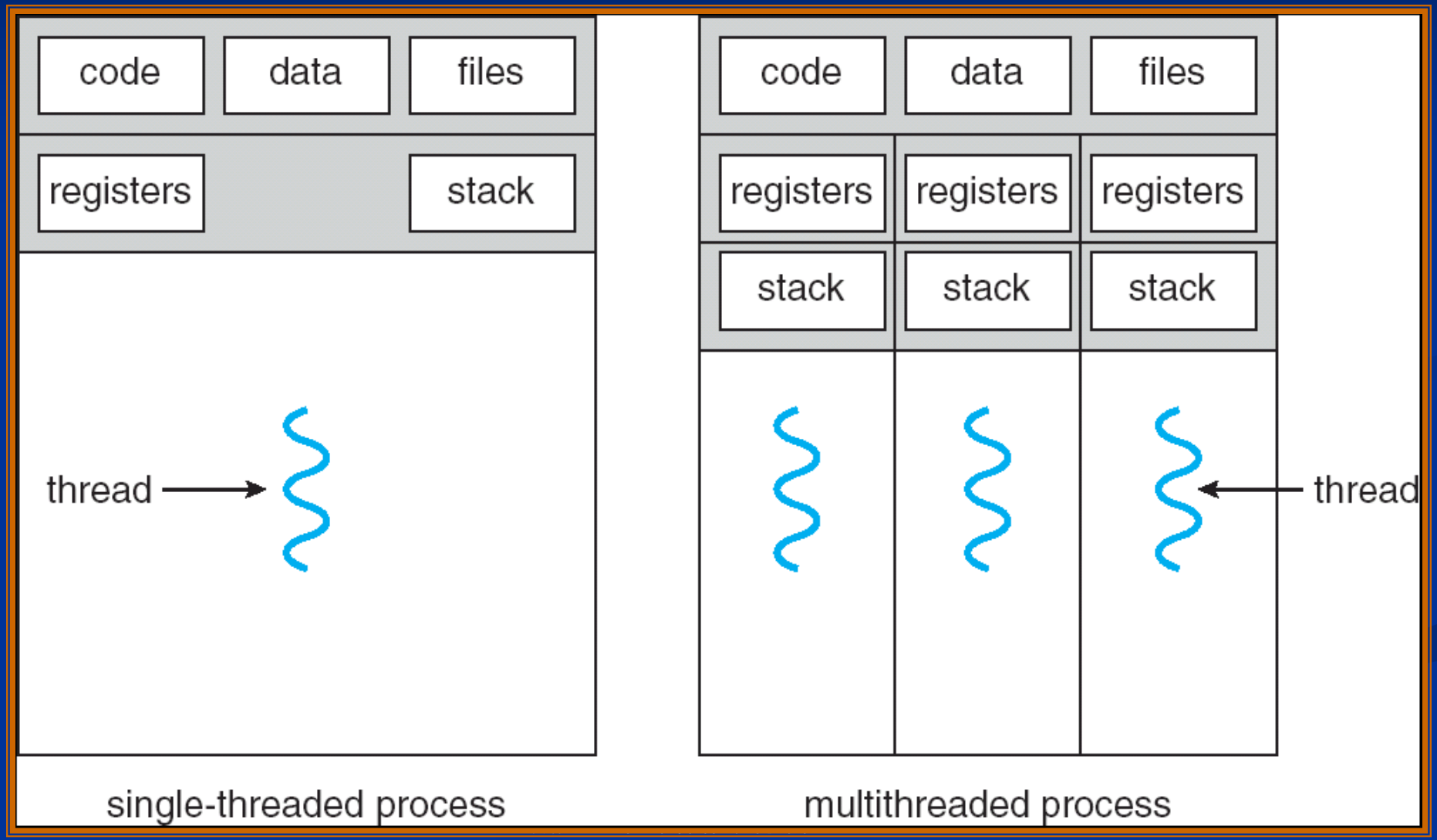
线程

- 线程概念
  - 有时称轻量级进程
  - 进程中的一个轻型运行实体
  - 是一个CPU调度和分派的基本单
  - 可并发执行
  - 共享进程资源



2.6 线程

(2)线程与进程的关系



2.6 线程(thread)

(3) 线程的开销

- 线程控制块（TCB)
- 寄存器状态，堆栈，局部变量拷贝
- 线程运行状态（就绪、阻塞、执行）
- 可以创建、撤消另一个线程
- 优先级与信号屏蔽

(4) 引入线程的好处

- 创建一个新线程花费时间少（结束亦如此）
- 两个线程的切换花费时间少
- 同一进程内的线程共享内存和文件，因此它们之间相互通信无须调用内核
- 适合多处理机系统

2.6 线程
2.6.2 线程的实现方式

- 内核支持线程
  - Windows 2000/XP、Macintosh 和OS/2 操作系统
- 用户级线程
  - 一些数据库管理系统如 infomix
- 两者结合方法
  - Solaris、Linux

2.6 线程
2.6.2 线程的实现方式

(1) 内核支持线程

- 内核支持线程，是在内核的支持下运行，即无论是用户进程中的线程，还是系统进程中的线程，他们的创建、撤消和切换等，也是依靠内核实现。
- 在内核空间还为每一个内核支持线程设置了一个线程控制块，内核是根据该控制块而感知某线程的存在的，并对其加以控制。
- 对于通常的进程，无论是系统进程还是用户进程，进程的创建、撤消，以及要求由系统设备完成的I/O操作，都是利用系统调用而进入内核，再由内核中的相应处理程序予以完成。进程的切换同样是在内核的支持下实现的。

2.6 线程
2.6.2 线程的实现方式

(1) 内核支持线程

- 若系统中设置的是内核支持线程，则以线程为单位进行调度。

- 在采用轮转法调度时，是各个线程轮流执行一个时间片。同样假定进程A中只有一个内核支持线程，而在进程B中有100个内核支持线程。此时进程B可以获得的CPU时间是进程A的100倍，且进程B可使100个系统调用并发工作。

2.6 线程
2.6.2 线程的实现方式

(2)内核支持线程

- 优点：
  - 对多处理器，核心可以同时调度同一进程的多个线程
  - 阻塞是在线程一级完成
  - 核心例程是多线程的
- 缺点：
  - 在同一进程内的线程切换调用内核，导致速度下降

2.6 线程
2.6.2 线程的实现方式

(2) 用户级线程

- 用户级线程仅存在于用户空间中。
- 对于这种线程的创建、撤消、线程之间的同步与通信等功能，都无须利用系统调用来实现。对于用户级线程的切换，通常是发生在一个应用进程的诸多线程之间，这时，也同样无须内核的支持。
- 可以为一个应用程序建立多个用户级线程。在一个系统中的用户级线程的数目，可以达到数百个至数千个由于这些线程的任务控制块都是设置在用户空间，而线程所执行的操作，也无须内核的帮助，因而内核完全不知道用户级线程的存在。

2.6 线程
2.6.2 线程的实现方式

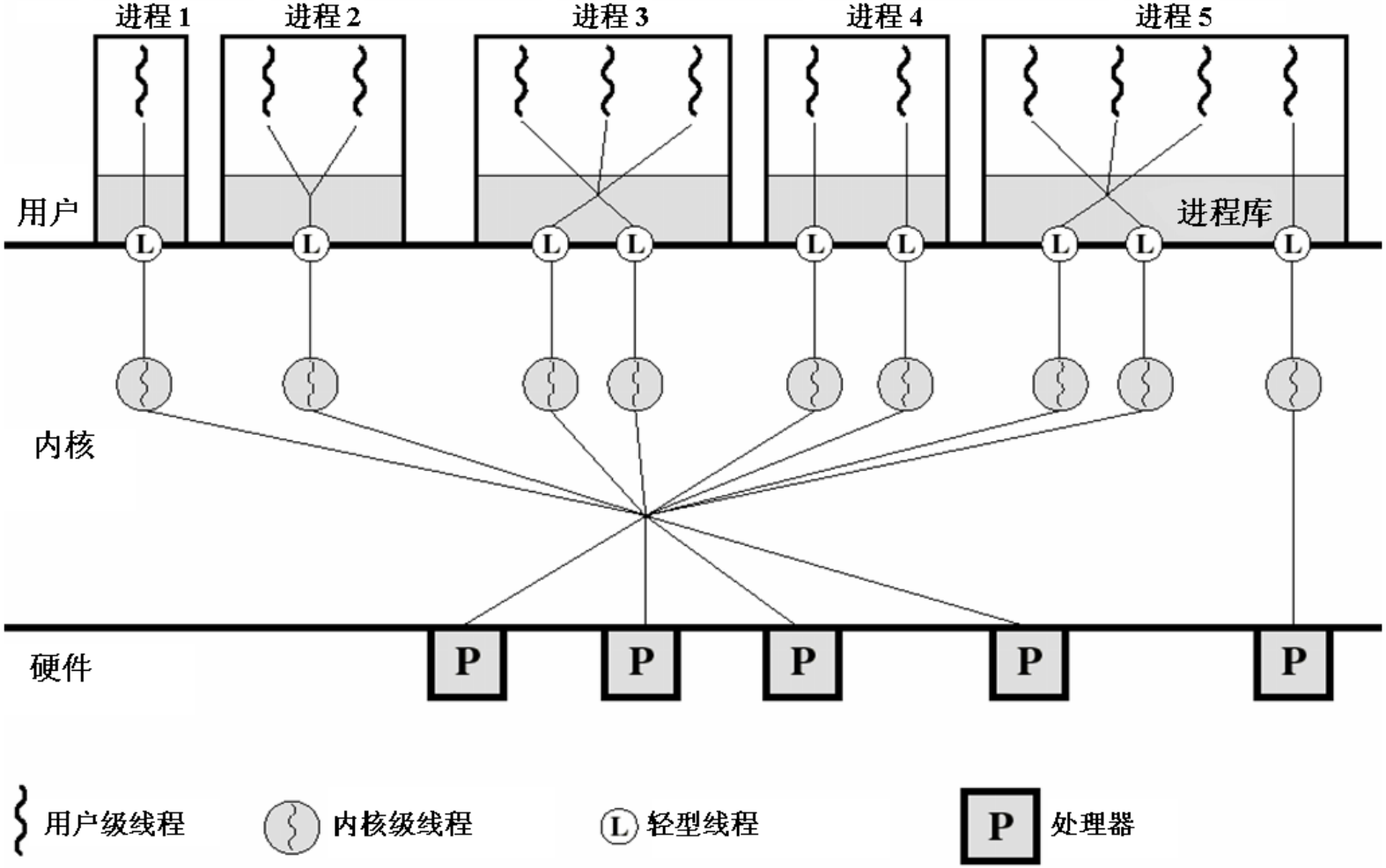
(2) 用户级线程

- 对于设置了用户级线程的系统，其调度仍是以进程为单位进行的。

- 在采用轮转调度算法时，各个进程轮流执行一个时间片。但假如在进程A中包含了一个用户级线程，而在另一个进程B中含有100个用户级线程，这样，进程A中线程的运行时间，将是进程B 中各线程运行时间的100倍，相应地，其速度要决上100倍.

(2) 用户级线程

- Solaris系统的线程机制



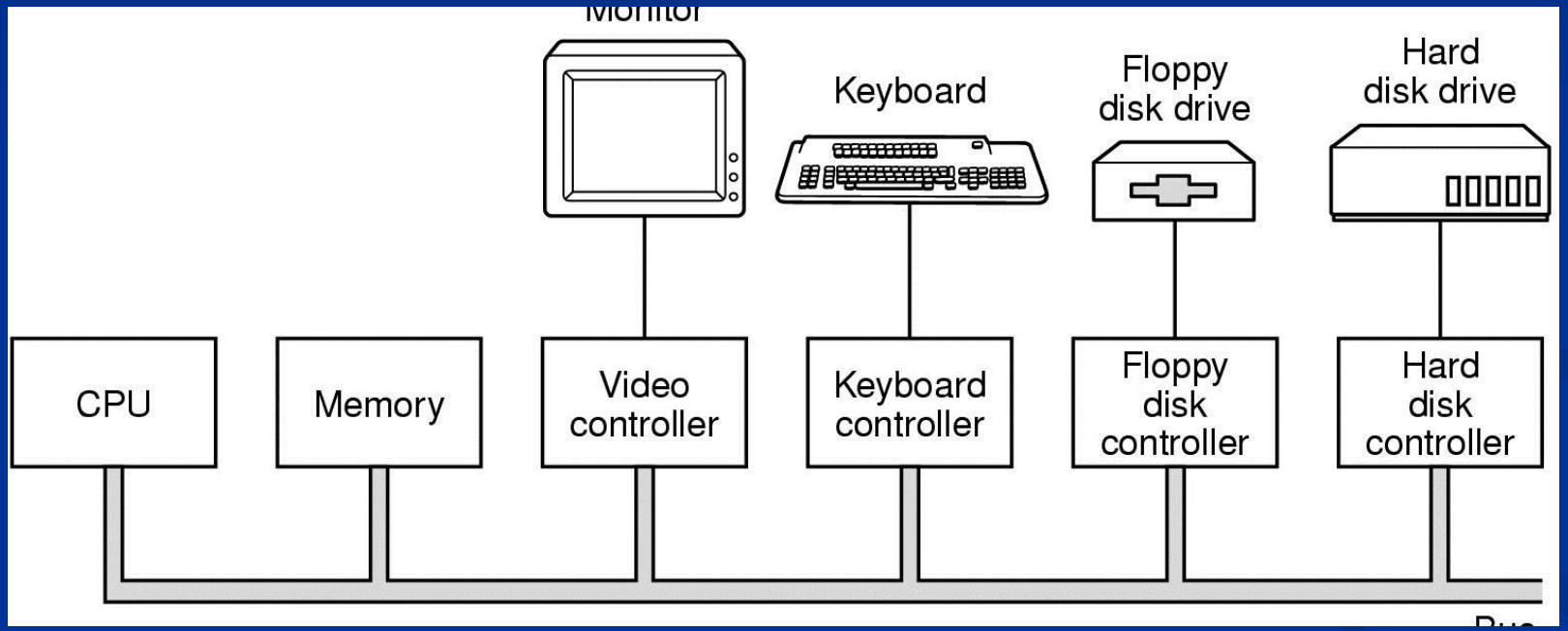
2.6 线程
2.6.2 线程的实现方式
(2) 用户级线程

- 优点：
  - 线程切换不调用核心
  - 调度是应用程序特定的：可以选择最好的算法
  - ULT可运行在任何操作系统上（只需要线程库）
- 缺点：
  - 大多数系统调用是阻塞的，因此核心阻塞进程，故进程中所有线程将被阻塞
  - 核心只将处理器分配给进程，同一进程中的两个线程不能同时运行于两个处理器上

补充材料：OS运行与应用程序运行

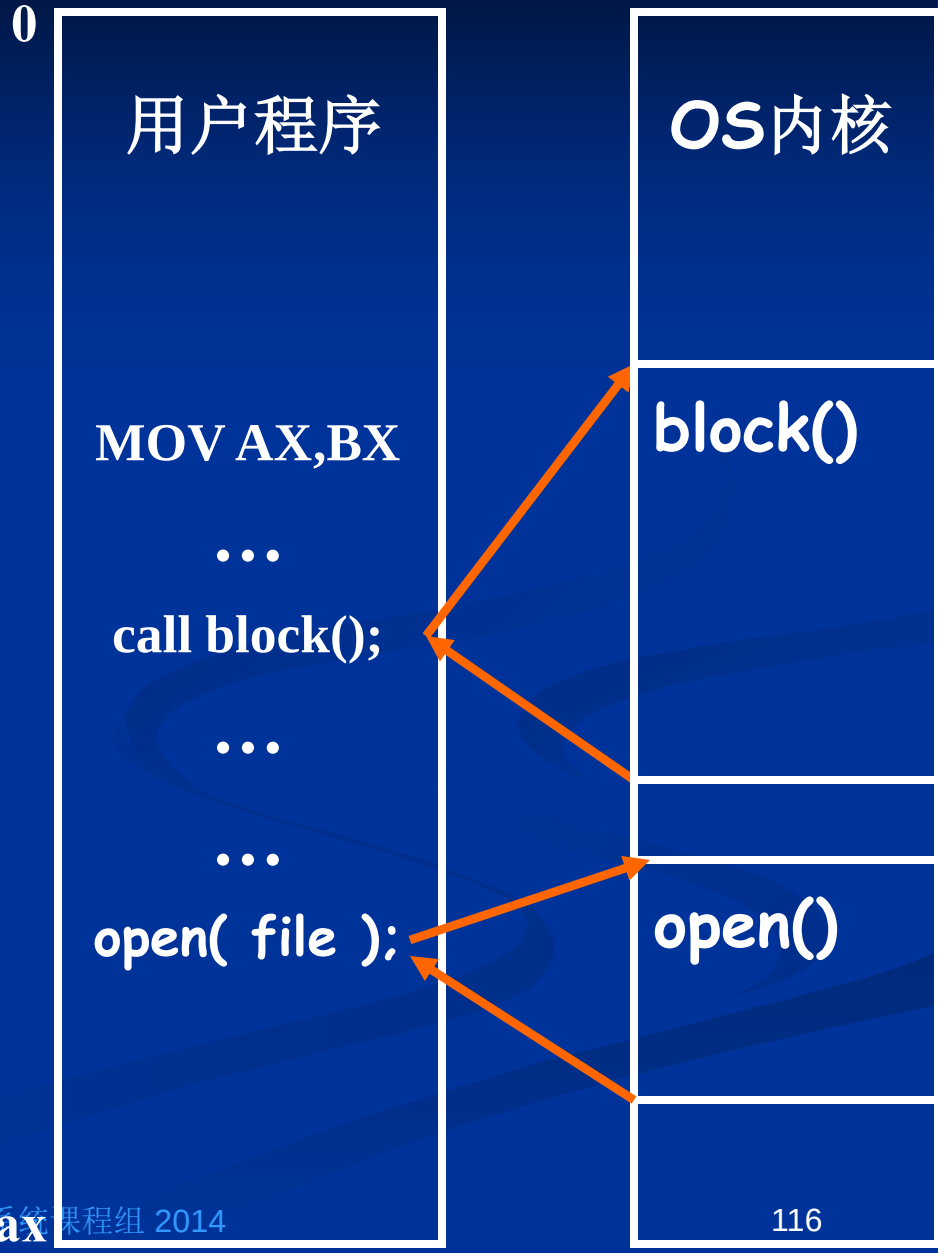
(1) 操作系统如何实现对计算机的控制

- 计算机系统架构



(1) 操作系统如何实现对计算机的控制

- 程序运行与操作系统运行
  - 操作系统初始化
  - 运行应用程序
  - 正常：普通代码
  - 特殊：调用OS代码



补充材料：OS的运行
(2) 指令系统

- 访问存储器指令：
  处理器和存储器间数据传送
- I/O指令：
  处理器和I/O模块间数据传送和命令发送
- 算术逻辑指令（数据处理指令）：
  执行数据算术和逻辑操作
- 控制转移指令：
  指定一个新的指令的执行起点
- 处理器控制指令：
  修改处理器状态，改变处理器工作方式

补充材料：OS的运行

(2) 指令系统：特权指令和非特权指令

- 特权指令：

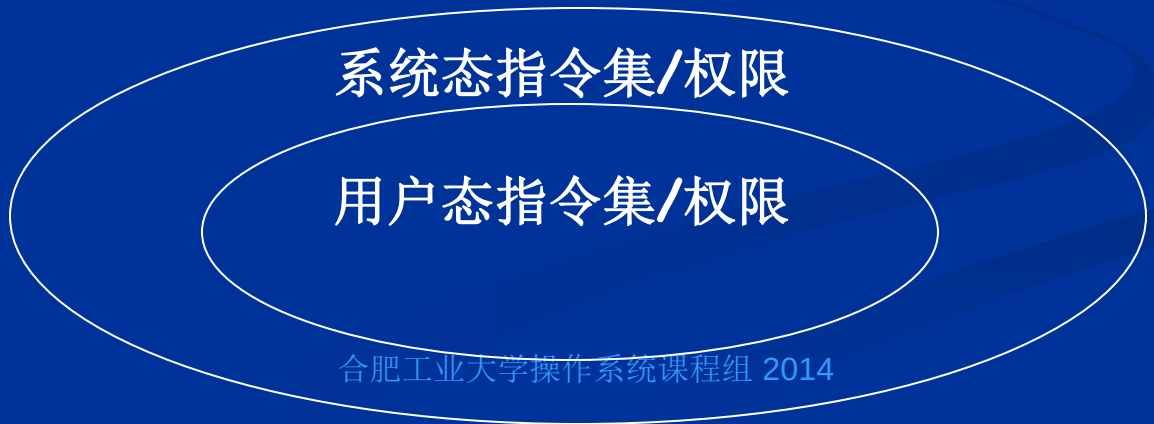
  只能由操作系统使用的指令

- 特权指令一般引起处理器状态的切换
  - 处理器通过特殊的机制将处理器状态切换到操作系统运行的特权状态（管态）
  - 然后将处理权移交给操作系统中的一段特殊代码，这一个过程称为陷入
- 使用多道程序设计技术的计算机指令系统必须要区分为特权指令和非特权指令

补充材料：OS的运行

(3) 系统态和用户态

- 根据运行程序对资源和机器指令的使用权限将处理器设置为不同状态
- 将处理器工作状态划分为系统态（管态）和用户态（目态）
  - 系统态：操作系统管理程序运行的状态，较高的特权级别，又称为特权态（特态）、核心态、管态；
  - 用户态：用户程序运行时的状态，较低的特权级别，又称为普通态（普态）、目态



(4) 实例：Intel x86环保护机制



- 4级：Ring0-Ring4;
- 各级有保护性检查（地址校验、I/O限制）

- 四个级别运行不同类程序：
  - Ring0-操作系统核心代码
  - Ring1-运行关键设备驱动程序和I/O处理例程
  - Ring2-其他受保护共享代码，如语言系统运行环境
  - Ring3-各种用户程序

- 现有基于x86处理器的操作系统，多数UNIX、Linux以及Windows系列只用了Ring0和Ring3两级

本章习题

- P68(修订版教材):
  2,4,6,8,11,17,18,23,25,26,28,33,36,41